geospatial-analytics: the Geospatial Analytics program — user guide

geospatial-analytics-ai999x 1.1.0, Rust build: command line, desktop app and browser. **Creator / engineer:** Ibnul Jaif Farabi · **Company:** AI999X · support@ai999x.com

Geospatial Analytics covers map projections and coordinates, geodesic distance and area, spatial statistics, remote-sensing indices, solar position and terrain, and urban and network analysis. This guide is about `geospatial-analytics`, the model as a self-contained program for Windows and Linux: 99 capabilities in 14 topics, answered on your own computer with no network, no Python and nothing else to install. The same compiled model runs in the AI999X desktop app and in the browser on the model's page (https://www.ai999x.com/models/geospatial-analytics-ai999x/); the Python package (`pip install geospatial-analytics-ai999x`) has its own guide. Every command output printed here is what the program printed.

Contents

- 1. Get it: desktop app, command line, browser, Python
- 2. Quick start
- 3. Writing inputs
- 4. Outcomes and exit codes
- 5. Scripts and agents: batch and JSON
- 6. Figures
- 7. In a web page or Node.js: WebAssembly
- 8. Every capability

1. Get it

Version 1.4.0 of the AI999X desktop app and of the command-line bundle. Checksums for every file: https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS; the newest version is always under https://downloads.ai999x.com/desktop/latest/. Full instructions: https://www.ai999x.com/desktop-cli/

what	system	download
Desktop app: Geospatial Analytics	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-geospatial-analytics-1.4.0-windows-x86_64.zip
Desktop app: Geospatial Analytics	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-geospatial-analytics-1.4.0-linux-x86_64.tar.gz
Desktop app: all models	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-windows-x86_64.zip
Desktop app: all models	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
Command line	Windows x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-windows-x86_64.zip
Command line	Linux x86-64	https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz

Desktop app

Geospatial Analytics in its own app: search, questions in words, input forms, live figures and explanations. Windows: Download ai999x-geospatial-analytics-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x-geospatial-analytics.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-geospatial-analytics-1.4.0-linux-
x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-geospatial-analytics-1.4.0-linux-x86_64.tar.gz
./ai999x-geospatial-analytics-1.4.0-linux-x86_64/ai999x-geospatial-analytics
```

It opens on this model: type a question or pick a capability, fill in the form and press Compute. The AI999X desktop app with every model works the same way (choose the model first). Windows: Download ai999x-1.4.0-windows-x86_64.zip, right-click it → Extract All, then open the folder and start ai999x.exe. If Windows SmartScreen asks, choose More info → Run anyway. Linux:

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
tar xzf ai999x-1.4.0-linux-x86_64.tar.gz
./ai999x-1.4.0-linux-x86_64/ai999x
```

Command line — Linux

The bundle holds one program per model; `geospatial-analytics` is this one. The checksum line should print OK.

```
curl -LO https://downloads.ai999x.com/desktop/1.4.0/ai999x-cli-1.4.0-linux-x86_64.tar.gz
curl -LO https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS
sha256sum -c --ignore-missing SHA256SUMS
mkdir -p ~/.local/share && tar xzf ai999x-cli-1.4.0-linux-x86_64.tar.gz -C ~/.local/share
echo 'export PATH="$HOME/.local/share/ai999x-cli-1.4.0-linux-x86_64:$PATH"' >> ~/.bashrc && source
~/.bashrc
geospatial-analytics --version
```

Command line — Windows (PowerShell)

The fourth line should print True.

```
$F = "ai999x-cli-1.4.0-windows-x86_64.zip"
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/$F" -OutFile $F
Invoke-WebRequest "https://downloads.ai999x.com/desktop/1.4.0/SHA256SUMS" -OutFile SHA256SUMS
(Get-FileHash $F -Algorithm SHA256).Hash.ToLower() -eq ((Select-String -SimpleMatch $F
SHA256SUMS).Line -split ' ')[0]
Expand-Archive $F -DestinationPath "$env:LOCALAPPDATA\AI999X" -Force
$D = "$env:LOCALAPPDATA\AI999X\ai999x-cli-1.4.0-windows-x86_64"; $env:Path += ";$D"
[Environment]::SetEnvironmentVariable("Path", [Environment]::GetEnvironmentVariable("Path", "User")
+ ";$D", "User")
geospatial-analytics --version
```

In your browser

Open https://www.ai999x.com/models/geospatial-analytics-ai999x/, choose a capability and press Compute in your browser: the same program runs on your device as WebAssembly; nothing is sent anywhere.

Python

```
pip install geospatial-analytics-ai999x
```

2. Quick start

2.1 Check the install

Prints the model, its version and that this is the Rust build.

```
$ geospatial-analytics --version
geospatial-analytics-ai999x 1.0.3 (Rust build, ax-engine 0.1.0)
```

2.2 List the capabilities

Every capability the program answers: its id (what you pass to solve) and its name.

```
$ geospatial-analytics list
geo_albers_equal_area_forward           Albers equal-area conic projection: coordinates and scale
factors
geo_albers_equal_area_inverse           Inverse Albers equal-area conic projection
geo_auxiliary_latitudes                 Auxiliary latitudes: geocentric, reduced, conformal,
authalic and rectifying
geo_bivariate_morans_i                  Bivariate Moran's I (spatial cross-correlation)
geo_brightness_temperature              At-sensor brightness temperature from thermal radiance
geo_classification_accuracy             Thematic map accuracy: overall accuracy, kappa, producer's
and user's accuracy
geo_cylindrical_equal_area_forward      Lambert cylindrical equal-area projection (sphere)
geo_d8_flow_accumulation                D8 flow accumulation of a small DEM
… 67 more
```

2.3 When an input is missing

No number is guessed: the answer names what is missing, shows a complete call, and the program exits with status 1.

```
$ geospatial-analytics solve geo_geodesic_destination lat1=52.2 lon1=0.12 azimuth=45.0
"distance=100 km"
INVALID_INPUT: unknown capability 'geo_geodesic_destination'
```

2.4 Many questions at once

One JSON request per line in, one JSON answer per line out, in order: for scripts, notebooks and agents (the program starts once). In PowerShell: Get-Content cases.jsonl | geospatial-analytics batch

`cases.jsonl`:

```
{"key": "geo_geodesic_destination", "inputs": {"lat1": 52.2, "lon1": 0.12, "azimuth": 45.0,
"distance": "100 km", "ellipsoid": "WGS84"}}
{"key": "geo_utm_forward", "inputs": {"lat": 51.5, "lon": -0.13, "ellipsoid": "WGS84"}}
```

```
$ geospatial-analytics batch < cases.jsonl
{"outcome":"INVALID_INPUT","message":"unknown capability
'geo_geodesic_destination'","value":null,"unit":null}
{"outcome":"SOLVED","message":"","value":699195.875854,"unit":"m","solved_for":"easting","si":
[["lat",51.5],["lon",-0.13]],"values":[["easting",699195.875854,"m"],
["northing",5709335.01926,"m"],["k",1.00008718886,""],["gamma",2.24681916539,""],["zone",30,""],
["hemisphere","N",""]],"target":"easting","phrase":"UTM easting","assumptions":["80 S <= latitude
<= 84 N"],"name":"Latitude and longitude to UTM coordinates (with scale factor and
convergence)","equation":"zone from the longitude (Norway and Svalbard exceptions), then transverse
Mercator by Krueger's series with k0 = 0.9996, false easting 500 km, false northing 10000 km in the
south"}
```

3. Writing inputs

Inputs are written `name=value`, one per argument; `geospatial-analytics info <capability>` lists the names. Quote an argument that holds a space or brackets (bash, zsh and PowerShell all accept double quotes). The answer always states each input as it was read, converted to SI, so a misread unit is visible at once.

kind	example	how it is read
a number with a unit	`"R=0.128 nm"`	any unit of the same kind; converted to SI
a number in SI	`R=1.28e-10`	no unit means the SI unit shown by info
a choice	`structure=FCC`	one of the values info lists
a list or vector	`"v=[1, 2, 3]" or "v=1, 2, 3"`	brackets keep it a list
a matrix	`"A=[[1, 2], [3, 4]]"`	one list per row
true / false	`inclusive=true`	for yes/no options

An equation can be solved for any of its quantities: give the others and add `--target <symbol>`. Without `--target`, the program solves for the quantity the capability is named after.

4. Outcomes and exit codes

The program never guesses a number: when it cannot answer, it says why and exits with status 1, so a script can tell an answer from a refusal without reading the text.

outcome	meaning	exit status
SOLVED	the answer: every output with its unit, the method, the inputs in SI, the conditions assumed	0
NEEDS_INPUT	an input is missing; the message names it and shows a complete call	1
INVALID_INPUT	a value or unit of the wrong kind, an unknown input name, or a value outside the conditions	1
AMBIGUOUS	the inputs allow two different answers; say which one is meant	1
OUT_OF_SCOPE	the model has no capability for this, or the result would not be meaningful	1
(usage)	a command the program does not know: it prints its usage	2

5. Scripts and agents: batch and JSON

`geospatial-analytics solve ... --json` prints one answer as one line of JSON. `geospatial-analytics batch` reads one request per line on standard input and writes one JSON answer per line, in order, starting the program once for any number of requests. A request is `{"key": "<capability>", "inputs": {"name": "value", ...}, "target": null}`. In PowerShell, pipe the file in: `Get-Content cases.jsonl | geospatial-analytics batch`. `geospatial-analytics catalog` prints every capability with its inputs as JSON, to build forms or tool definitions.

answer field	meaning
outcome	one of the outcomes above
message	why there is no answer (empty when SOLVED)
value, unit	the main result in SI units
values	every output: [symbol, value, unit] (procedures)
target	the quantity solved for
si	each numeric input converted to SI: [symbol, value]
assumptions	the conditions the answer relies on
name, equation	the capability and its equation or method

```
import json, subprocess

reqs = [{"key": "<capability>", "inputs": {"name": "value"}}]
out = subprocess.run(["geospatial-analytics", "batch"], input="\n".join(map(json.dumps, reqs)),
                     capture_output=True, text=True, check=True).stdout
answers = [json.loads(line) for line in out.splitlines()]
```

6. Figures

`geospatial-analytics figures <capability> name=value ... --out DIR` draws the live figures for those inputs and writes each as an SVG file into DIR (an existing folder); without `--out` the SVG text comes back as JSON. The desktop app and the model page draw the same figures as soon as you press Compute.

7. In a web page or Node.js: WebAssembly

The same model is a WebAssembly module, `geospatial_analytics_ai999x.wasm` (https://www.ai999x.com/compute/rust/geospatial_analytics_ai999x.wasm), with a small loader, `ax-wasm.js` (https://www.ai999x.com/compute/rust/ax-wasm.js). It runs in a browser, a Web Worker or Node.js, with no server.

```
<script src="ax-wasm.js"></script>
<script>
  AXWasm.load("geospatial_analytics_ai999x.wasm").then(function (m) {
    var a = m.solve({key: "<capability>", inputs: {name: "value"}});
    console.log(a.outcome, a.values || a.value);
    console.log(m.catalog().capabilities.length + " capabilities");
  });
</script>
```

8. Every capability

By topic. For each: what it does, what it needs and gives, when it applies, and a command with what the program printed for it. `geospatial-analytics info <id>` shows the same inputs and outputs in the terminal.

Cartography (2)

Ground distance from a map distance and a representative fraction — `geo_map_scale_ground_distance`

The real-world distance represented by a distance measured on a map of scale 1 : scale.

$$D = d \times S$$

symbol		meaning	SI unit
`map_distance`	in	distance measured on the map	m
`scale`	in	scale denominator (1 : scale)	—
`ground_distance`	out	ground distance	m
`ground_km`	out	ground distance in kilometres	—

Applies when: positive map distance and scale.

```
$ geospatial-analytics solve geo_map_scale_ground_distance "map_distance=4.2 cm" scale=50000
ground_distance = 2100 m
ground_km = 2.1
  by: Ground distance from a map distance and a representative fraction  (ground distance = map
distance x scale denominator)
  map_distance = 0.042 m   []
  scale = 50000   []
  assumes: positive map distance and scale
```

What the worked example means: This calculation determines the real-world distance represented by a measurement taken on a map. Multiplying the measured map distance of 4.2 cm by the scale denominator of 50000 shows that the actual ground distance is 2100 m. This result can also be expressed as 2.1 for easier interpretation in kilometers.

Map scale (representative fraction) from a map and a ground distance — `geo_map_scale_from_distances`

The scale denominator and representative fraction of a map on which a known ground distance measures a given map distance.

$$S = \frac{D}{d}, \quad RF = \frac{1}{S}$$

symbol		meaning	SI unit
`map_distance`	in	distance on the map	m
`ground_distance`	in	corresponding ground distance	m
`scale`	out	scale denominator S (scale 1 : S)	—

symbol		meaning	SI unit
`rf`	out	representative fraction 1/S	—

Applies when: ground distance > map distance > 0.

```
$ geospatial-analytics solve geo_map_scale_from_distances "map_distance=5 cm" "ground_distance=2.5
km"
scale = 50000
rf = 2e-05
  by: Map scale (representative fraction) from a map and a ground distance  (S = ground distance /
map distance in the same unit)
  map_distance = 0.05 m   []
  ground_distance = 2500 m   []
  assumes: ground distance > map distance > 0
```

What the worked example means: This calculation determines the map's scale by comparing the measured distance on the map to the corresponding actual ground distance. Given a map distance of 5 cm and a ground distance of 2.5 km, the resulting scale denominator is 50000. This indicates that one unit on the map represents 50000 units on the ground, expressed as a representative fraction of 2e-05.

Coordinate notation (5)

Decimal degrees to degrees, minutes and seconds — `geo_decimal_to_dms`

A signed latitude or longitude in decimal degrees written as whole degrees, whole minutes, seconds and a hemisphere letter.

$$d = \lfloor |x| \rfloor, \quad m = \lfloor 60(|x| - d) \rfloor, \quad s = 3600\left(|x| - d - \tfrac{m}{60}\right)$$

symbol		meaning	SI unit
`decimal`	in	signed angle (decimal degrees)	—
`axis`	in	whether the angle is a latitude (N/S) or a longitude (E/W)	—
`degrees`	out	whole degrees	—
`minutes`	out	whole minutes	—
`seconds`	out	seconds	—
`hemisphere`	out	N, S, E or W	—

Applies when: |latitude| <= 90, |longitude| <= 180.

```
$ geospatial-analytics solve geo_decimal_to_dms decimal=51.477222 axis=latitude
degrees = 51
minutes = 28
seconds = 37.9992
hemisphere = N
  by: Decimal degrees to degrees, minutes and seconds  (exact decimal arithmetic: degrees =
floor(|x|), minutes = floor(60 frac), seconds = the remainder times 3600)
  decimal = 51.477222   []
  axis = latitude
  assumes: |latitude| <= 90, |longitude| <= 180
```

What the worked example means: This calculation transforms a single signed decimal value representing geographic position into traditional Degrees, Minutes, and Seconds notation. It takes the input angle 51.4772 associated with the latitude coordinate system and decomposes it into whole degrees 51, remaining minutes 28, and fractional seconds 37.9992. The sign of the original value determines the hemisphere letter N, indicating whether the location is North, South, East, or West relative to the equator or prime meridian.

Decode a geohash: cell centre and size — `geo_geohash_decode`

The centre latitude and longitude of a geohash cell and its half-height and half-width in degrees.

$$[\varphi_{min}, \varphi_{max}] \times [\lambda_{min}, \lambda_{max}] \text{ by alternate bisection}$$

symbol		meaning	SI unit
`geohash`	in	geohash string (characters 0-9, b-z without a, i, l, o)	—
`lat_center`	out	latitude of the cell centre	—
`lon_center`	out	longitude of the cell centre	—
`lat_error`	out	half the cell height (degrees)	—
`lon_error`	out	half the cell width (degrees)	—

Applies when: 1 to 12 valid characters.

```
$ geospatial-analytics solve geo_geohash_decode geohash=u4pruydqqvj
lat_center = 57.64911063
lon_center = 10.40743969
lat_error = 6.705522537e-07
lon_error = 6.705522537e-07
  by: Decode a geohash: cell centre and size  (each character gives five bits that halve the
longitude and latitude intervals alternately (longitude first))
  geohash = u4pruydqqvj
  assumes: 1 to 12 valid characters
```

What the worked example means: This process decodes the provided geohash string to determine the geographic location it represents. The algorithm iteratively halves the global coordinate intervals based on the binary values of each character, starting with longitude. The final result provides the precise center coordinates as 57.6491 and 10.4074, along with the uncertainty bounds defined by the half-height 6.70552e-07 and half-width 6.70552e-07. These values describe the exact spatial footprint of the cell within the standard geohash grid system.

Degrees, minutes and seconds to decimal degrees — `geo_dms_to_decimal`

A latitude or longitude written in degrees, minutes and seconds with a hemisphere letter, converted to signed decimal degrees and radians.

$$\varphi = \pm\left(d + \frac{m}{60} + \frac{s}{3600}\right)$$

symbol		meaning	SI unit
`degrees`	in	whole degrees (>= 0)	—

symbol		meaning	SI unit
`minutes`	in	whole minutes (0-59)	—
`seconds`	in	seconds (0 <= s < 60)	—
`hemisphere`	in	hemisphere letter (S and W are negative)	—
`decimal`	out	signed decimal degrees	—
`radians`	out	signed angle in radians	—

Applies when: 0 <= minutes <= 59; 0 <= seconds < 60; at most 90 (N/S) or 180 (E/W) degrees.

```
$ geospatial-analytics solve geo_dms_to_decimal degrees=51 minutes=28 seconds=38 hemisphere=N
decimal = 51.47722222
radians = 0.8984470176
  by: Degrees, minutes and seconds to decimal degrees  (decimal = sign (degrees + minutes/60 +
seconds/3600), exact rational arithmetic)
  seconds = 38   []
  hemisphere = N
  assumes: 0 <= minutes <= 59; 0 <= seconds < 60; at most 90 (N/S) or 180 (E/W) degrees
```

What the worked example means: This calculation transforms a geographic position expressed in the traditional sexagesimal system into a single continuous number suitable for digital mapping. By summing the whole degrees with the fractional parts contributed by the minutes and seconds, and applying a negative sign if the hemisphere is South or West, the model produces a signed value of 51.4772. This result allows legacy survey data to be integrated directly into modern computational systems that require standard numerical inputs for latitude and longitude.

Geohash of a point and its cell — `geo_geohash_encode`

The geohash of a latitude and longitude at a given number of characters, with the centre and half-sizes of the geohash cell.

$$\mathrm{bits} = \lambda_1\varphi_1\lambda_2\varphi_2\ldots, \quad \mathrm{character} = \mathrm{base32}(5\ \mathrm{bits})$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`precision`	in	number of geohash characters (1-12)	—
`geohash`	out	geohash	—
`lat_center`	out	latitude of the cell centre	—
`lon_center`	out	longitude of the cell centre	—
`lat_error`	out	half the cell height (degrees)	—
`lon_error`	out	half the cell width (degrees)	—

Applies when: 1 to 12 characters.

```
$ geospatial-analytics solve geo_geohash_encode lat=57.64911 lon=10.40744 precision=11
geohash = u4pruydqqvj
lat_center = 57.64911063
lon_center = 10.40743969
lat_error = 6.705522537e-07
lon_error = 6.705522537e-07
  by: Geohash of a point and its cell  (interleaved binary bisection of longitude and latitude
(longitude first), five bits per base-32 character)
  lat = 57.64911   []
  lon = 10.40744   []
  assumes: 1 to 12 characters
```

What the worked example means: This calculation determines the geohash string for a specific geographic location defined by input latitude 57.6491 and longitude 10.4074 using a precision of 11 characters. The resulting identifier is u4pruydqqvj, which corresponds to a rectangular grid cell on the map. The center of this cell is located at latitude 57.6491 and longitude 10.4074, while its half-height is 6.70552e-07 degrees and its half-width is 6.70552e-07 degrees, defining the spatial resolution of the index.

MGRS grid reference of a point (1 m) — `geo_mgrs_encode`

The Military Grid Reference System (MGRS / USNG) reference of a WGS84 latitude and longitude to 1 m: zone, latitude band, 100 km square letters and 5-digit easting and northing.

$$\mathrm{MGRS} = z\,B\,C_{100k}R_{100k}\,\lfloor E\rfloor \bmod 10^5\,\lfloor N\rfloor \bmod 10^5$$

symbol		meaning	SI unit
`lat`	in	latitude (WGS84) (decimal degrees)	—
`lon`	in	longitude (WGS84) (decimal degrees)	—
`mgrs`	out	MGRS reference (no spaces)	—

Applies when: 80 S <= latitude <= 84 N (polar regions use UPS).

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation converts the input geographic coordinates, specifically latitude 50 and longitude 10, into a Military Grid Reference System string. The resulting value, 32UNA7166639109, represents a precise location on the Earth's surface to a resolution of one meter. It combines the zone number, latitude band letter, 100 km square identifiers, and truncated easting and northing digits into a single alphanumeric code that is easier to communicate than raw decimal degrees.

Coordinate systems and conversions (7)

Auxiliary latitudes: geocentric, reduced, conformal, authalic and rectifying — `geo_auxiliary_latitudes`

The five auxiliary latitudes of a geodetic latitude on the WGS84 or GRS80 ellipsoid (used by the sphere-based forms of map projections and geodesics).

$$\tan\psi = (1-e^2)\tan\varphi,\ \ \tan\beta = (1-f)\tan\varphi,\ \chi = \mathrm{gd}\,\psi_{iso},\ \xi = \arcsin\frac{q}{q_p},\ \mu = \frac{\pi}{2}\frac{m(\varphi)}{m(\pi/2)}$$

symbol		meaning	SI unit
`lat`	in	geodetic latitude (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`geocentric`	out	geocentric latitude (degrees)	—
`reduced`	out	reduced (parametric) latitude (degrees)	—
`conformal`	out	conformal latitude (degrees)	—
`authalic`	out	authalic latitude (degrees)	—
`rectifying`	out	rectifying latitude (degrees)	—

Applies when: latitude strictly between -90 and 90.

```
$ geospatial-analytics solve geo_auxiliary_latitudes lat=45 ellipsoid=WGS84
geocentric = 44.80757678
reduced = 44.90378785
conformal = 44.80768406
authalic = 44.87170287
rectifying = 44.85568199
  by: Auxiliary latitudes: geocentric, reduced, conformal, authalic and rectifying  (tan(psi) = (1-
e^2) tan(phi); tan(beta) = (1-f) tan(phi); chi = atan(sinh(isometric latitude)); xi = asin(q/q_p);
mu = (pi/2) m(phi)/m(pi/2))
  lat = 45   []
  ellipsoid = WGS84
  assumes: latitude strictly between -90 and 90
```

What the worked example means: This calculation determines five specific auxiliary latitudes derived from the input geodetic latitude of 45 using the reference ellipsoid WGS84. These values, which include the geocentric latitude 44.8076, reduced latitude 44.9038, conformal latitude 44.8077, authalic latitude 44.8717, and rectifying latitude 44.8557, serve as mathematical bridges between the ellipsoidal Earth model and spherical approximations. They are essential for simplifying complex map projections and geodesic calculations by transforming ellipsoidal coordinates into forms that behave predictably on a sphere.

Geocentric (ECEF) to geodetic coordinates — `geo_ecef_to_geodetic`

Geodetic latitude, longitude and ellipsoidal height of a point given by its Earth-centred, Earth-fixed coordinates X, Y, Z on the WGS84 or GRS80 ellipsoid.

$$\varphi = \operatorname{atan2}\left(Z + e^2 N \sin\varphi,\ \sqrt{X^2 + Y^2}\right), \quad h = p\cos\varphi + Z\sin\varphi - \frac{a^2}{N}$$

symbol		meaning	SI unit
`X`	in	geocentric X	m
`Y`	in	geocentric Y	m
`Z`	in	geocentric Z	m
`ellipsoid`	in	reference ellipsoid	—
`lat`	out	geodetic latitude (degrees)	—
`lon`	out	longitude (degrees)	—
`h`	out	ellipsoidal height	m

Applies when: 6200 km <= distance from the centre <= 100000 km; not on the polar axis.

```
$ geospatial-analytics solve geo_ecef_to_geodetic "X=4517590.879 m" "Y=796576.168 m" "Z=4487348.409
m" ellipsoid=WGS84
lat = 44.55996486
lon = 10.000037
h = 49468.66699 m
  by: Geocentric (ECEF) to geodetic coordinates  (fixed-point iteration phi = atan2(Z + e^2 N
sin(phi), p) to convergence; h = p cos(phi) + Z sin(phi) - a^2/N)
  X = 4517590.879 m   []
  Y = 796576.168 m   []
  Z = 4487348.409 m   []
  ellipsoid = WGS84
  assumes: 6200 km <= distance from the centre <= 100000 km; not on the polar axis
```

What the worked example means: This calculation converts Earth-centered, Earth-fixed coordinates into geographic positions. Using the input values X of 4.51759e+06 m, Y of 796576 m, and Z of 4.48735e+06 m on the specified reference ellipsoid, the algorithm determines the location's position on the globe. The resulting geodetic latitude is 44.56, indicating north-south position, while the longitude is 10, indicating east-west position. The ellipsoidal height is 49468.7 m, which represents the distance from the mathematical surface of the ellipsoid to the point.

Geodetic to geocentric (ECEF) coordinates — `geo_geodetic_to_ecef`

Earth-centred, Earth-fixed Cartesian coordinates X, Y, Z of a point given by geodetic latitude, longitude and ellipsoidal height on the WGS84 or GRS80 ellipsoid.

$$X = (N + h)\cos\varphi\cos\lambda,\ Y = (N + h)\cos\varphi\sin\lambda,\ Z = \left(N(1 - e^2) + h\right)\sin\varphi$$

symbol		meaning	SI unit
`lat`	in	geodetic latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`h`	in	ellipsoidal height	m
`ellipsoid`	in	reference ellipsoid	—
`X`	out	X (towards latitude 0, longitude 0)	m
`Y`	out	Y (towards longitude 90 E)	m
`Z`	out	Z (towards the north pole)	m

Applies when: latitude between -90 and 90; height between -100 km and 100000 km.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation converts the geodetic position defined by latitude 45, longitude 10, and ellipsoidal height 100 m into Earth-Centered, Earth-Fixed coordinates. The resulting values 4.44903e+06 m, 784484 m, and 4.48742e+06 m represent the point's location in a three-dimensional Cartesian system relative to the center of mass of the reference ellipsoid, enabling precise spatial analysis and satellite navigation applications.

Geodetic to local east-north-up (ENU) coordinates — `geo_geodetic_to_enu`

Local tangent-plane coordinates east, north and up of a point relative to an origin (both given by latitude, longitude and ellipsoidal height), and the straight-line (slant) range.

$$\begin{bmatrix} e \\ n \\ u \end{bmatrix} = \begin{bmatrix} -\sin\lambda_0 & \cos\lambda_0 & 0 \\ -\sin\varphi_0\cos\lambda_0 & -\sin\varphi_0\sin\lambda_0 & \cos\varphi_0 \\ \cos\varphi_0\cos\lambda_0 & \cos\varphi_0\sin\lambda_0 & \sin\varphi_0 \end{bmatrix} \begin{bmatrix} \Delta X \\ \Delta Y \\ \Delta Z \end{bmatrix}$$

symbol		meaning	SI unit
`lat0`	in	latitude of the local origin (decimal degrees)	—
`lon0`	in	longitude of the local origin (decimal degrees)	—
`h0`	in	ellipsoidal height of the origin	m
`lat`	in	latitude of the point (decimal degrees)	—
`lon`	in	longitude of the point (decimal degrees)	—
`h`	in	ellipsoidal height of the point	m
`ellipsoid`	in	reference ellipsoid	—
`e`	out	east coordinate	m
`n`	out	north coordinate	m
`u`	out	up coordinate	m
`slant_range`	out	straight-line distance from the origin	m

Applies when: origin not at a pole.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation converts geodetic positions into a local tangent plane frame centered at the origin. Using the reference ellipsoid WGS84, the system computes the straight-line vector from the origin at latitude 46.017, longitude 7.75 and height 1673 m to the target at latitude 45.976, longitude 7.658 and height 4531 m. The resulting east coordinate is -7134.76 m, the north coordinate is -4556.32 m, and the vertical up component is 2852.39 m. These values represent the displacement in meters relative to the local horizontal plane, while the total straight-line distance between the two points is given as the slant range 8933.14 m.

Local east-north-up (ENU) to geodetic coordinates — `geo_enu_to_geodetic`

Latitude, longitude and ellipsoidal height of a point given by its local east, north and up coordinates relative to an origin on the WGS84 or GRS80 ellipsoid.

$$\mathbf{X} = \mathbf{X}_0 + R^T(\varphi_0, \lambda_0)\,[e\ n\ u]^T$$

symbol		meaning	SI unit
`lat0`	in	latitude of the local origin (decimal degrees)	—
`lon0`	in	longitude of the local origin (decimal degrees)	—
`h0`	in	ellipsoidal height of the origin	m
`e`	in	east coordinate	m
`n`	in	north coordinate	m

symbol		meaning	SI unit
`u`	in	up coordinate	m
`ellipsoid`	in	reference ellipsoid	—
`lat`	out	latitude (degrees)	—
`lon`	out	longitude (degrees)	—
`h`	out	ellipsoidal height	m

Applies when: origin not at a pole; the point not deep inside the Earth.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the global geographic coordinates of a target point by transforming its local east, north, and up distances relative to a specified origin. Using the origin's latitude 46.017, longitude 7.75, and height 1673 m, along with the input offsets of -7123.1 m meters east, -4557.3 m meters north, and 2848.2 m meters up, the method rotates these local vectors into an Earth-centered frame and converts them back to geodetic form. The resulting output provides the absolute latitude 45.976, longitude 7.65815, and ellipsoidal height 4526.8 m, allowing local survey data to be integrated into global mapping systems.

Look angles: azimuth, elevation and range from an observer to a target — `geo_look_angles`

The azimuth (clockwise from true north), elevation angle above the local horizontal plane and slant range from an observer to a target, both given by latitude, longitude and ellipsoidal height (a satellite, aircraft or summit).

$$A = \operatorname{atan2}(e, n), \quad \varepsilon = \operatorname{atan2}\left(u, \sqrt{e^2 + n^2}\right)$$

symbol		meaning	SI unit
`lat0`	in	observer latitude (decimal degrees)	—
`lon0`	in	observer longitude (decimal degrees)	—
`h0`	in	observer ellipsoidal height	m
`lat`	in	target latitude (decimal degrees)	—
`lon`	in	target longitude (decimal degrees)	—
`h`	in	target ellipsoidal height	m
`ellipsoid`	in	reference ellipsoid	—
`azimuth`	out	azimuth (degrees clockwise from north)	—
`elevation`	out	elevation angle (degrees, negative below the horizon)	—
`slant_range`	out	straight-line distance	m

Applies when: observer not at a pole; target distinct from the observer.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the look angles from an observer to a target using their respective global coordinates. Converting both positions into a local East-North-Up coordinate system relative to the observer gives the vector pointing to the target. The resulting azimuth of 172.243 indicates the direction clockwise from true north, while the elevation of 43.4643 shows the angle above or

below the local horizon. The straight-line distance between the two points is found to be 3.75137e+07 m. These values are essential for orienting antennas, cameras, or tracking systems toward the object.

UTM zone, latitude band and central meridian — `geo_utm_zone`

The Universal Transverse Mercator zone number (with the Norway and Svalbard exceptions), the MGRS latitude band letter, the hemisphere and the zone's central meridian for a latitude and longitude.

$$z = \left\lfloor \frac{\lambda + 180^\circ}{6^\circ} \right\rfloor + 1, \quad \lambda_0 = 6z - 183^\circ$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`zone`	out	UTM zone number (1-60)	—
`band`	out	latitude band letter (C-X)	—
`hemisphere`	out	N or S	—
`central_meridian`	out	central meridian of the zone (degrees)	—

Applies when: 80 S <= latitude <= 84 N.

```
$ geospatial-analytics solve geo_utm_zone lat=51.5 lon=-0.13
zone = 30
band = U
hemisphere = N
central_meridian = -3
  by: UTM zone, latitude band and central meridian  (zone = floor((lon + 180)/6) + 1 with zone 32
widened over south-west Norway and zones 31/33/35/37 over Svalbard; band = 8-degree letter from 80
S (X spans 72-84 N))
  lat = 51.5   []
  lon = -0.13   []
  assumes: 80 S <= latitude <= 84 N
```

What the worked example means: This calculation determines the Universal Transverse Mercator grid zone designation for the given coordinates. Based on an input longitude of -0.13 and latitude of 51.5, the system assigns zone 30 and latitude band U. The hemisphere is identified as N, and the longitudinal center of this specific zone is calculated to be -3. This standardizes the geographic position into a rectangular coordinate system used for mapping, accounting for special exceptions in Norway and Svalbard.

Ellipsoidal geodesy (9)

Arc length along a parallel of latitude — `geo_parallel_arc`

The distance along a parallel of latitude between two longitudes (the shorter way round) on the WGS84 or GRS80 ellipsoid, and the radius of the parallel.

$$s = N(\varphi)\cos\varphi\,|\Delta\lambda|, \quad N = \frac{a}{\sqrt{1 - e^2\sin^2\varphi}}$$

symbol		meaning	SI unit
`lat`	in	latitude of the parallel (decimal degrees)	—
`lon1`	in	first longitude (decimal degrees)	—
`lon2`	in	second longitude (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`distance`	out	arc length along the parallel	m
`radius`	out	radius of the parallel N cos(phi)	m

Applies when: latitude strictly between -90 and 90; different longitudes.

```
$ geospatial-analytics solve geo_parallel_arc lat=60 lon1=0 lon2=1 ellipsoid=WGS84
distance = 55800.00157 m
radius = 3197104.587 m
  by: Arc length along a parallel of latitude  (radius of the parallel N(phi) cos(phi) times the
longitude difference in radians)
  lat = 60   []
  lon1 = 0   []
  lon2 = 1   []
  ellipsoid = WGS84
  assumes: latitude strictly between -90 and 90; different longitudes
```

What the worked example means: This calculation determines the distance traveled along a line of constant latitude, known as a parallel, on the reference ellipsoid. By using the latitude 60 and the longitude difference between 0 and 1, the method computes the radius of that specific parallel and multiplies it by the angular separation in radians. The resulting arc length is 55800 m, which represents the east-west span between the two points at this high or low latitude. The effective radius of the circle formed by this parallel is calculated as 3.1971e+06 m, showing how the curvature of the Earth affects distances away from the equator.

Area and perimeter of a geodesic polygon on the ellipsoid — `geo_geodesic_polygon_area`

The area enclosed by a polygon whose edges are geodesics on the WGS84 or GRS80 ellipsoid, and its perimeter (vertices given as latitude and longitude lists, in either orientation).

$$S = -\oint \frac{a^2}{2}[q(\varphi) - q(\varphi_c)]d\lambda, \quad q(\varphi) = (1-e^2)\left[\frac{\sin\varphi}{1-e^2\sin^2\varphi} + \frac{\operatorname{artanh}(e\sin\varphi)}{e}\right]$$

symbol		meaning	SI unit
`lats`	in	latitudes of the vertices in order (decimal degrees)	—
`lons`	in	longitudes of the vertices in the same order (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`area`	out	enclosed area on the ellipsoid	m^2
`perimeter`	out	perimeter (sum of the geodesic edge lengths)	m

Applies when: a simple polygon not enclosing a pole; vertices within 88 degrees of the equator; edges at most 90 degrees long.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the true surface area and perimeter of a geodesic polygon defined by its vertices. The area, reported as 1.23088e+10 m^2 square meters, represents the enclosed region on the reference ellipsoid, while the perimeter, given as 443771 m meters, is the sum of the shortest paths along the curved surface connecting each vertex. These values are derived using Green's theorem adapted for the ellipsoidal geometry, ensuring accuracy for land management or boundary analysis where flat-earth approximations fail.

Area of a latitude-longitude cell on the ellipsoid — `geo_graticule_cell_area`

The area of the region between two parallels and two meridians (a graticule cell or grid square) on the WGS84 or GRS80 ellipsoid, and its share of the whole ellipsoid's surface.

$$A = \frac{a^2}{2}[q(\varphi_2) - q(\varphi_1)](\lambda_2 - \lambda_1)$$

symbol		meaning	SI unit
`lat1`	in	southern latitude (decimal degrees)	—
`lat2`	in	northern latitude (decimal degrees)	—
`lon1`	in	western longitude (decimal degrees)	—
`lon2`	in	eastern longitude (east of lon1) (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`area`	out	area of the cell	m^2
`fraction_of_ellipsoid`	out	area as a fraction of the ellipsoid's total surface	—

Applies when: lat1 < lat2; 0 < lon2 - lon1 <= 360.

```
$ geospatial-analytics solve geo_graticule_cell_area lat1=0 lat2=1 lon1=0 lon2=1 ellipsoid=WGS84
area = 1.230846389e+10 m^2
fraction_of_ellipsoid = 2.413113797e-05
  by: Area of a latitude-longitude cell on the ellipsoid  (authalic function: area = (a^2/2)
[q(phi2) - q(phi1)] (lambda2 - lambda1))
  lat1 = 0   []
  lat2 = 1   []
  lon1 = 0   []
  lon2 = 1   []
  ellipsoid = WGS84
  assumes: lat1 < lat2; 0 < lon2 - lon1 <= 360
```

What the worked example means: This calculation determines the true surface area of a graticule cell on an ellipsoid using the authalic function. By integrating between the southern latitude 0 and northern latitude 1, and across the longitudes from 0 to 1, the method accounts for the curvature of the Earth. The resulting area of 1.23085e+10 m^2 reflects how grid cells shrink in size as they move away from the equator, providing a precise measure of physical extent rather than a distorted planar approximation.

Destination along a rhumb line (loxodrome) on the ellipsoid — `geo_rhumb_line_destination`

The point reached by travelling a given distance on a constant bearing (rhumb line) from a start point on the WGS84 or GRS80 ellipsoid.

$$m_2 = m_1 + s\cos\alpha, \quad \Delta\lambda = \tan\alpha\,(\psi_2 - \psi_1)$$

symbol		meaning	SI unit
`lat1`	in	latitude of the start point (decimal degrees)	—
`lon1`	in	longitude of the start point (decimal degrees)	—
`bearing`	in	constant bearing, clockwise from north (decimal degrees)	—
`distance`	in	distance travelled	m
`ellipsoid`	in	reference ellipsoid	—
`lat2`	out	latitude of the destination (degrees)	—
`lon2`	out	longitude of the destination (degrees)	—

Applies when: the line stays more than 1 degree from the poles; distance > 0.

```
$ geospatial-analytics solve geo_rhumb_line_destination lat1=50 lon1=-5 bearing=30 "distance=500
km" ellipsoid=WGS84
lat2 = 53.89168147
lon2 = -1.361213633
  by: Destination along a rhumb line (loxodrome) on the ellipsoid  (meridian distance advances by s
cos(bearing) (Helmert's series, inverted by Newton steps); the longitude by tan(bearing) times the
change of isometric latitude)
  lat1 = 50   []
  lon1 = -5   []
  bearing = 30   []
  distance = 500000 m   []
  ellipsoid = WGS84
  assumes: the line stays more than 1 degree from the poles; distance > 0
```

What the worked example means: This calculation determines the destination point reached by moving a specified distance on a constant compass bearing across an ellipsoidal Earth model. Starting from latitude 50 and longitude -5, the method advances the meridian distance by 500 km times the cosine of the bearing, while adjusting longitude based on the change in isometric latitude. The resulting position is latitude 53.8917 and longitude -1.36121, representing a path that maintains a steady heading rather than following the shortest great-circle route.

Geodesic destination point on the ellipsoid (direct problem) — `geo_geodesic_destination`

The point reached by travelling a given distance along a geodesic from a start point at a given initial azimuth on the WGS84 or GRS80 ellipsoid, and the azimuth of the geodesic on arrival.

$$\frac{s}{b} = \int_{\sigma_1}^{\sigma_2} \sqrt{1 + k^2\sin^2\sigma}\,d\sigma, \quad \lambda_{12} = \omega_{12} - f\sin\alpha_0 \int_{\sigma_1}^{\sigma_2} \frac{2 - f}{1 + (1 - f)\sqrt{1 + k^2\sin^2\sigma}}\,d\sigma$$

symbol		meaning	SI unit
`lat1`	in	latitude of the start point (decimal degrees)	—
`lon1`	in	longitude of the start point (decimal degrees)	—
`azimuth`	in	initial azimuth, clockwise from north (decimal degrees)	—

symbol		meaning	SI unit
`distance`	in	geodesic distance travelled	m
`ellipsoid`	in	reference ellipsoid	—
`lat2`	out	latitude of the destination (degrees)	—
`lon2`	out	longitude of the destination (degrees)	—
`azimuth2`	out	forward azimuth at the destination (degrees clockwise from north)	—

Applies when: start latitude strictly between -90 and 90; 0 < distance <= 15000 km.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the precise location reached after traveling a geodesic distance of 100 km from a starting point at 52.2 latitude and 0.12 longitude. The path follows an initial bearing of 45 on the specified ellipsoid. The resulting destination coordinates are 52.8308 and 1.16915, and the forward azimuth at that point is 45.8325. This method accounts for the Earth's flattening, providing higher accuracy than spherical models for long-distance navigation.

Meridian arc length between two latitudes — `geo_meridian_arc`

The distance along a meridian of the WGS84 or GRS80 ellipsoid between two latitudes, and the meridian distance of each latitude from the equator.

$$m(\varphi) = \frac{a}{1+n}\left[\left(1+\frac{n^2}{4}+\frac{n^4}{64}\right)\varphi - \frac{3}{2}\left(n-\frac{n^3}{8}\right)\sin 2\varphi + \frac{15}{16}\left(n^2-\frac{n^4}{4}\right)\sin 4\varphi - \frac{35}{48}n^3\sin 6\varphi + \frac{315}{512}n^4 \mathrm{s}\right.$$

symbol		meaning	SI unit
`lat1`	in	first latitude (decimal degrees)	—
`lat2`	in	second latitude (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`distance`	out	meridian arc length between the latitudes	m
`m1`	out	meridian distance from the equator to lat1 (negative south)	m
`m2`	out	meridian distance from the equator to lat2 (negative south)	m

Applies when: latitudes between -90 and 90 and different.

```
$ geospatial-analytics solve geo_meridian_arc lat1=0 lat2=90 ellipsoid=WGS84
distance = 10001965.73 m
m1 = 0 m
m2 = 10001965.73 m
  by: Meridian arc length between two latitudes  (Helmert's series for the meridian distance in the
third flattening n (to n^4))
  lat1 = 0   []
  lat2 = 90   []
  ellipsoid = WGS84
  assumes: latitudes between -90 and 90 and different
```

What the worked example means: This calculation determines the distance along a line of longitude between two specified latitudes using Helmert's series expansion for the third flattening. The result, 1.0002e+07 m, represents the surface distance on the reference ellipsoid defined by WGS84. Additionally, the tool provides the individual meridian distances from the equator to each latitude, reported as 0 m for the first location and 1.0002e+07 m for the second. These values are essential for understanding true ground distances in geodetic applications where flat map projections introduce distortion.

Midpoint of a geodesic on the ellipsoid — `geo_geodesic_midpoint`

The point halfway along the shortest path (geodesic) between two points on the WGS84 or GRS80 ellipsoid, with the geodesic length.

$$P_{mid} = \mathrm{direct}\left(P_1,\ \alpha_1,\ s_{12}/2\right)$$

symbol		meaning	SI unit
`lat1`	in	latitude of the first point (decimal degrees)	—
`lon1`	in	longitude of the first point (decimal degrees)	—
`lat2`	in	latitude of the second point (decimal degrees)	—
`lon2`	in	longitude of the second point (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`lat_mid`	out	latitude of the midpoint (degrees)	—
`lon_mid`	out	longitude of the midpoint (degrees)	—
`distance`	out	length of the geodesic between the two points	m

Applies when: distinct points at most 150 degrees apart; latitudes strictly between -90 and 90.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the geographic center of the shortest path connecting two points on an ellipsoid. Using the specified reference ellipsoid, it computes the azimuth and distance from the first point to the second, then advances halfway along that path to find the midpoint. The resulting coordinates are 52.385 degrees north and -41.2786 degrees east, while the full geodesic length is 5.58581e+06 m meters.

Radii of curvature of the ellipsoid at a latitude — `geo_radii_of_curvature`

The meridional radius of curvature M, the prime-vertical radius of curvature N and the Gaussian mean radius sqrt(MN) of the WGS84 or GRS80 ellipsoid at a latitude.

$$M = \frac{a(1-e^2)}{(1-e^2\sin^2\varphi)^{3/2}},\quad N = \frac{a}{\sqrt{1-e^2\sin^2\varphi}},\quad R_G = \sqrt{MN}$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—

symbol		meaning	SI unit
`M`	out	meridional radius of curvature	m
`N`	out	prime-vertical radius of curvature	m
`R_gauss`	out	Gaussian mean radius sqrt(M N)	m

Applies when: latitude between -90 and 90.

```
$ geospatial-analytics solve geo_radii_of_curvature lat=45 ellipsoid=WGS84
M = 6367381.816 m
N = 6388838.29 m
R_gauss = 6378101.03 m
  by: Radii of curvature of the ellipsoid at a latitude  (M = a(1-e^2)/(1-e^2 sin^2 phi)^(3/2), N =
a/sqrt(1-e^2 sin^2 phi))
  lat = 45   []
  ellipsoid = WGS84
  assumes: latitude between -90 and 90
```

What the worked example means: This calculation determines the local curvature of the reference ellipsoid at a specific latitude. Using the semi-major axis and eccentricity defined by the WGS84, it computes the meridional radius of curvature, which describes how much the surface curves north-south, and the prime-vertical radius, which describes east-west curvature. The result for latitude 45 yields a meridional radius of 6.36738e+06 m and a prime-vertical radius of 6.38884e+06 m. These values are essential for transforming geodetic coordinates into Cartesian systems and for designing accurate map projections.

Rhumb-line (loxodrome) distance and bearing on the ellipsoid — `geo_rhumb_line_inverse`

The constant bearing and the length of the rhumb line (loxodrome) between two points on the WGS84 or GRS80 ellipsoid, the shorter way round in longitude.

$$\tan\alpha = \frac{\Delta\lambda}{\Delta\psi}, \quad s = \frac{\Delta m}{\cos\alpha}, \quad \psi = \operatorname{arsinh}(\tan\varphi) - e\operatorname{artanh}(e\sin\varphi)$$

symbol		meaning	SI unit
`lat1`	in	latitude of the start point (decimal degrees)	—
`lon1`	in	longitude of the start point (decimal degrees)	—
`lat2`	in	latitude of the end point (decimal degrees)	—
`lon2`	in	longitude of the end point (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`distance`	out	rhumb-line length	m
`bearing`	out	constant bearing (degrees clockwise from north)	—

Applies when: latitudes within 89 degrees of the equator; distinct points.

```
$ geospatial-analytics solve geo_rhumb_line_inverse lat1=50 lon1=-5 lat2=58 lon2=3 ellipsoid=WGS84
distance = 1032440.869 m
bearing = 30.40644034
  by: Rhumb-line (loxodrome) distance and bearing on the ellipsoid  (bearing = atan2(dlambda, dpsi)
with the isometric latitude psi; length = dm / cos(bearing) with Helmert's meridian distance (N
cos(phi) |dlambda| along a parallel))
  lat1 = 50   []
  lon1 = -5   []
  lat2 = 58   []
  lon2 = 3   []
  ellipsoid = WGS84
  assumes: latitudes within 89 degrees of the equator; distinct points
```

What the worked example means: This calculation determines the length and direction of a loxodrome between two points on the ellipsoid. Using the start coordinates 50 and -5, and the end coordinates 58 and 3, the method computes the difference in longitude relative to the difference in isometric latitude. The resulting constant heading is 30.4064, and the total path length along this line is 1.03244e+06 m. This represents the course a vessel or aircraft would steer to maintain a fixed angle with all meridians, which differs from the shortest great-circle route.

Facility location (3)

Closest facility for every node of a network — `geo_closest_facility`

For every node of a road network, the nearest facility by shortest-path distance and that distance, with the mean distance to the nearest facility.

$$f^*(i) = \arg\min_{f \in F} d(i, f)$$

symbol		meaning	SI unit
`edges`	in	undirected network as weighted edges between nodes numbered 0 .. n-1	—
`facilities`	in	facility nodes	—
`nearest`	out	nearest facility of each node	—
`distance`	out	distance to it	—
`mean_distance`	out	mean distance to the nearest facility	—

Applies when: every node reaches a facility; no node equally near two facilities.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the nearest service point for every location in the network by measuring shortest path distances. For each node, it identifies the specific facility that minimizes travel cost and records that minimum distance. The resulting mean distance of 1.4 provides a single metric for overall accessibility across the entire network, indicating how far an average user must travel to reach the closest available option among the defined facilities.

Maximal covering location problem (MCLP) on a network — `geo_maximal_covering_location`

The largest demand that p facilities, chosen from candidate nodes, can cover within a coverage distance on a road network, and the covered share.

$$\max_{S \subseteq C, |S|=p} \sum_i w_i \mathbf{1}\left[\min_{j \in S} d_{ij} \le s\right]$$

symbol		meaning	SI unit
`edges`	in	undirected network as weighted edges between nodes numbered 0 .. n-1	—
`demand`	in	demand at nodes 0 .. n-1	—
`candidates`	in	candidate facility nodes	—
`p`	in	number of facilities	—
`coverage`	in	coverage distance (network distance <= coverage counts)	—
`covered`	out	maximum covered demand	—
`fraction`	out	covered share of the total demand	—

Applies when: at most 20000 candidate combinations.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This analysis identifies the best locations for 2 facilities from the candidate set to maximize service coverage within a distance of 3. By evaluating all feasible combinations, the model determines that these specific placements can serve 95 units of demand. This result indicates that 0.904762 of the total demand is accessible within the specified threshold, highlighting the efficiency of the chosen network locations.

p-median facility location on a network — `geo_p_median`

The p facility sites, chosen from candidate nodes, that minimize the total demand-weighted shortest-path distance from every node to its nearest open facility.

$$\min_{S \subseteq C, |S|=p} \sum_i w_i \min_{j \in S} d_{ij}$$

symbol		meaning	SI unit
`edges`	in	undirected network as weighted edges between nodes numbered 0 .. n-1	—
`demand`	in	demand at nodes 0 .. n-1	—
`candidates`	in	candidate facility nodes	—
`p`	in	number of facilities to open	—
`objective`	out	minimum total weighted distance	—
`facilities`	out	chosen facility nodes (ascending)	—
`mean_distance`	out	demand-weighted mean distance	—

Applies when: connected network; at most 20000 candidate combinations; a unique optimal set.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This analysis determines the best locations for a fixed number of facilities by evaluating all possible combinations from the candidate nodes. Using shortest-path distances on the connected network, it minimizes the sum of demand-weighted distances from every node to its nearest open facility. The result identifies the specific sites [2, 5] that yield the lowest total cost of 150, with an average demand-weighted distance of 1.42857.

Geostatistics (5)

Empirical (Matheron) semivariogram in distance bins — `geo_empirical_semivariogram`

The classical (Matheron) empirical semivariogram of sampled values: half the mean squared difference of all pairs whose separation falls in each distance bin, and the pair counts.

$$\hat{\gamma}(h_k) = \frac{1}{2N_k} \sum_{(i,j)\in k} (z_i - z_j)^2$$

symbol		meaning	SI unit
`x`	in	x coordinates of the samples	—
`y`	in	y coordinates of the samples	—
`z`	in	sample values	—
`bin_edges`	in	increasing distance bin edges (a bin includes its lower edge)	—
`gamma`	out	semivariance of each bin	—
`counts`	out	number of pairs in each bin	—

Applies when: every bin contains at least one pair; bins [e_k, e_k+1).

```
$ geospatial-analytics solve geo_empirical_semivariogram "x=[0, 1, 2, 3, 4]" "y=[0, 0, 0, 0, 0]"
"z=[1, 3, 2, 5, 4]" "bin_edges=[0, 1.5, 2.5, 4.5]"
gamma = [1.875,1.5,4.33333333333]
counts = [4.0,3.0,3.0]
  by: Empirical (Matheron) semivariogram in distance bins  (gamma(h_k) = 1/(2 N_k) sum over pairs
in bin k of (z_i - z_j)^2)
  assumes: every bin contains at least one pair; bins [e_k, e_k+1)
```

What the worked example means: This calculation determines the empirical semivariogram by averaging half the squared differences of sample values within specific distance bins. For the first bin defined by edges starting at [0, 1.5, 2.5, 4.5], the computed semivariance is [1.875, 1.5, 4.33333] based on [4, 3, 3] unique pairs of samples. This value quantifies the average dissimilarity between points separated by distances falling within that range, serving as a fundamental measure of spatial autocorrelation.

Fit an exponential or Gaussian variogram model (nugget, partial sill, range) — `geo_variogram_model_fit`

The nugget, partial sill, range and sill of an exponential or Gaussian variogram model fitted by ordinary least squares to an empirical semivariogram (lags and semivariances).

$$\gamma(h) = c_0 + c\left(1 - e^{-3h/a}\right) \text{ or } c_0 + c\left(1 - e^{-(7h/4a)^2}\right), \quad \min_{c_0,c,a} \sum_k \left(\hat{\gamma}_k - \gamma(h_k)\right)^2$$

symbol		meaning	SI unit
`h`	in	lag distances	—
`gamma`	in	semivariances at the lags	—
`model`	in	variogram model (pykrige parameterization, practical range)	—
`nugget`	out	nugget c0	—
`psill`	out	partial sill c	—
`range`	out	practical range a	—
`sill`	out	sill c0 + c	—

Applies when: the least-squares range has an interior minimum (between min(h)/20 and 20 max(h)).

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the optimal parameters for an exponential or Gaussian variogram model by minimizing the difference between the theoretical curve and the empirical semivariogram data. The result provides the nugget effect -0.0628821, which represents micro-scale variation or measurement error, and the partial sill 7.16435, indicating the variance of spatially correlated data. The practical range 11.3285 defines the distance beyond which spatial correlation becomes negligible, while the total sill 7.10147 marks the plateau value of the semivariance.

Inverse distance weighted (IDW) interpolation at a point — `geo_idw_interpolation`

The inverse-distance-weighted (Shepard) estimate of a variable at a location from sample points, with a stated power (2 is the usual choice).

$$\hat{z}(\mathbf{p}) = \frac{\sum_i d_i^{-p} z_i}{\sum_i d_i^{-p}}$$

symbol		meaning	SI unit
`x`	in	x coordinates of the samples	—
`y`	in	y coordinates of the samples	—
`z`	in	sample values	—
`px`	in	x of the location	—
`py`	in	y of the location	—
`power`	in	distance power p (> 0)	—
`estimate`	out	interpolated value	—

Applies when: power > 0 (at most 10); a location on a sample returns that sample's value.

```
$ geospatial-analytics solve geo_idw_interpolation "x=[0, 10, 0, 10]" "y=[0, 0, 10, 10]" "z=[1, 2,
3, 4]" px=2 py=3 power=2
estimate = 1.65897994
  by: Inverse distance weighted (IDW) interpolation at a point  (z = sum d_i^-p z_i / sum d_i^-p
over all samples)
  px = 2   []
  py = 3   []
  power = 2   []
  assumes: power > 0 (at most 10); a location on a sample returns that sample's value
```

What the worked example means: This calculation determines an estimated value of 1.65898 for the variable at a specific location defined by coordinates 2 and 3. The estimate is derived from sample points located at coordinates [0, 10, 0, 10] and [0, 0, 10, 10] with observed values [1, 2, 3, 4], using a distance power of 2. This method assumes that closer samples exert greater influence on the prediction than distant ones, producing a weighted average that reflects local spatial trends.

Ordinary kriging estimate and kriging variance at a point — `geo_ordinary_kriging`

The ordinary kriging estimate of a variable at a location from samples, with a given variogram model (nugget, sill, range), and the kriging variance.

$$\begin{bmatrix} \Gamma & \mathbf{1} \\ \mathbf{1}^T & 0 \end{bmatrix} \begin{bmatrix} \boldsymbol{\lambda} \\ \mu \end{bmatrix} = \begin{bmatrix} \boldsymbol{\gamma}_0 \\ 1 \end{bmatrix}, \quad \hat{z} = \boldsymbol{\lambda}^T \mathbf{z}, \quad \sigma^2_{OK} = \boldsymbol{\lambda}^T \boldsymbol{\gamma}_0 + \mu$$

symbol		meaning	SI unit
`x`	in	x coordinates of the samples	—
`y`	in	y coordinates of the samples	—
`z`	in	sample values	—
`px`	in	x of the location	—
`py`	in	y of the location	—
`model`	in	variogram model (pykrige parameterization)	—
`nugget`	in	nugget	—
`sill`	in	sill (nugget + partial sill)	—
`range`	in	range (practical range for exponential and gaussian)	—
`estimate`	out	kriging estimate	—
`variance`	out	kriging variance	—

Applies when: distinct sample locations; sill > nugget >= 0, range > 0; the variogram is 0 at zero distance (exact interpolation).

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the best linear unbiased prediction for a spatial variable at an unsampled location using ordinary kriging. By solving the kriging system with the provided sample coordinates, values, and variogram parameters including nugget, sill, and range, the method computes weights that minimize estimation variance while ensuring unbiasedness. The resulting estimate of 2.6048 represents the predicted value at the target location defined by its x and y coordinates, while the

variance of 0.877115 quantifies the uncertainty associated with this prediction based on the spatial structure defined by the model.

Polynomial trend surface (first or second order) by least squares — `geo_trend_surface`

The coefficients and R^2 of a first-order (plane) or second-order (quadratic) trend surface fitted to sampled values by least squares.

$$z = b_0 + b_1 x + b_2 y \ \left[+ b_3 x^2 + b_4 xy + b_5 y^2 \right]$$

symbol		meaning	SI unit
`x`	in	x coordinates	—
`y`	in	y coordinates	—
`z`	in	values	—
`order`	in	order of the surface (1 or 2)	—
`coefficients`	out	b0, b1 (x), b2 (y)[, b3 (x^2), b4 (xy), b5 (y^2)]	—
`r_squared`	out	coefficient of determination	—

Applies when: more samples than coefficients; values not all equal.

```
$ geospatial-analytics solve geo_trend_surface "x=[0, 1, 2, 0, 1, 2]" "y=[0, 0, 0, 1, 1, 1]" "z=[1,
3, 5.2, 2, 4.1, 6]" order=1
coefficients = [1.01666666667,2.05,0.966666666667]
r_squared = 0.9987204095
  by: Polynomial trend surface (first or second order) by least squares  (ordinary least squares by
the normal equations)
  assumes: more samples than coefficients; values not all equal
```

What the worked example means: This analysis computes a polynomial trend surface using ordinary least squares on the input coordinates and values. The resulting coefficients, such as [1.01667, 2.05, 0.966667], define the mathematical plane or quadratic surface that best approximates the spatial data. The coefficient of determination, 0.99872, quantifies how well this global trend explains the variation in the observed values.

Map projections (22)

Albers equal-area conic projection: coordinates and scale factors — `geo_albers_equal_area_forward`

Easting and northing of a point in the ellipsoidal Albers equal-area conic projection with two standard parallels, with the scale factors along the parallel (k) and the meridian (h = 1/k).

$$n = \frac{m_1^2 - m_2^2}{q_2 - q_1}, \quad \rho = \frac{a\sqrt{C - nq}}{n}, \quad x = FE + \rho \sin\theta,\ y = FN + \rho_0 - \rho\cos\theta,\ k = \frac{\rho n}{am} = \frac{1}{h}$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`lat1`	in	first standard parallel (decimal degrees)	—

symbol		meaning	SI unit
`lat2`	in	second standard parallel (decimal degrees)	—
`lat0`	in	latitude of origin (decimal degrees)	—
`lon0`	in	central meridian (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`FE`	in	false easting (default 0) (optional)	m
`FN`	in	false northing (default 0) (optional)	m
`x`	out	easting	m
`y`	out	northing	m
`k`	out	scale factor along the parallel	—
`h`	out	scale factor along the meridian	—

Applies when: parallels within 89 degrees of the equator and not symmetric about it.

```
$ geospatial-analytics solve geo_albers_equal_area_forward lat=40 lon=-100 lat1=29.5 lat2=45.5
lat0=23 lon0=-96 ellipsoid=GRS80
x = -338390.5876 m
y = 1894100.14 m
k = 0.9909686029
h = 1.009113707
  by: Albers equal-area conic projection: coordinates and scale factors  (n = (m1^2 - m2^2)/(q2 -
q1), C = m1^2 + n q1, rho = a sqrt(C - n q)/n with the authalic function q; x = FE + rho
sin(theta), y = FN + rho0 - rho cos(theta))
  lat = 40   []
  lon = -100   []
  lat1 = 29.5   []
  lat2 = 45.5   []
  lat0 = 23   []
  lon0 = -96   []
  ellipsoid = GRS80
  assumes: parallels within 89 degrees of the equator and not symmetric about it
```

What the worked example means: This calculation determines the projected coordinates for a specific location using the Albers equal-area conic projection. Given an input latitude of 40 and longitude of -100, along with standard parallels at 29.5 and 45.5, the resulting easting is -338391 m meters and the northing is 1.8941e+06 m meters. The scale factor along the parallel, denoted as k, is 0.990969, while the meridian scale factor h is 1.00911. These values allow for accurate area-preserving mapping by converting geographic coordinates into planar distances relative to the defined central meridian -96 and origin latitude 23.

Azimuthal equidistant projection (ellipsoidal, geodesic) — `geo_azimuthal_equidistant_forward`

Easting and northing of a point in the ellipsoidal azimuthal equidistant projection about a centre: the geodesic distance from the centre plotted at the geodesic azimuth (used for range rings and airline maps).

$$x = FE + s_{12}\sin\alpha_1, \quad y = FN + s_{12}\cos\alpha_1$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—

symbol		meaning	SI unit
`lon`	in	longitude (decimal degrees)	—
`lat0`	in	latitude of the centre (decimal degrees)	—
`lon0`	in	longitude of the centre (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`FE`	in	false easting (default 0) (optional)	m
`FN`	in	false northing (default 0) (optional)	m
`x`	out	easting	m
`y`	out	northing	m
`distance`	out	geodesic distance from the centre	m
`azimuth`	out	geodesic azimuth from the centre (degrees)	—

Applies when: point distinct from the centre and at most 150 degrees from it; centre within 89 degrees of the equator.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the planar coordinates for an ellipsoidal azimuthal equidistant projection centered at latitude 51.5 and longitude -0.13. The target point, located at latitude 40.71 and longitude -74.01, is plotted such that its map distance from the center equals the true geodesic distance on the Earth's surface. The resulting easting is -5.30106e+06 m and the northing is 1.76067e+06 m, derived from a radial distance of 5.58581e+06 m measured along the initial bearing of 288.373 degrees.

Equirectangular (equidistant cylindrical / plate carree) projection — `geo_equirectangular_forward`

x and y of a point in the equirectangular (equidistant cylindrical) projection of a sphere of radius R with a standard parallel (lat_ts = 0 is the plate carree).

$$x = R(\lambda - \lambda_0)\cos\varphi_{ts}, \quad y = R(\varphi - \varphi_0)$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`lat_ts`	in	standard parallel (true scale) (decimal degrees)	—
`lat0`	in	latitude of origin (decimal degrees)	—
`lon0`	in	central meridian (decimal degrees)	—
`R`	in	radius of the sphere	m
`x`	out	x	m
`y`	out	y	m

Applies when: sphere of radius R > 0.

```
$ geospatial-analytics solve geo_equirectangular_forward lat=45 lon=10 lat_ts=0 lat0=0 lon0=0
"R=6371000 m"
x = 1111949.266 m
y = 5003771.699 m
  by: Equirectangular (equidistant cylindrical / plate carree) projection  (x = R (lambda -
lambda0) cos(lat_ts), y = R (phi - phi0))
  lat = 45   []
  lon = 10   []
  lat_ts = 0   []
  lat0 = 0   []
  lon0 = 0   []
  R = 6371000 m   []
  assumes: sphere of radius R > 0
```

What the worked example means: This calculation determines the planar coordinates for a specific location on a spherical model of the Earth. The horizontal position, or x, is derived from the difference between the point's longitude and the central meridian, scaled by the sphere's radius and adjusted for the standard parallel. The vertical position, or y, comes directly from the difference between the point's latitude and the origin latitude, multiplied by the radius. For this instance, the resulting x value is 1.11195e+06 m and the y value is 5.00377e+06 m, representing distances in meters from the projection center.

Gnomonic projection (sphere) — `geo_gnomonic_forward`

x and y of a point in the gnomonic projection (great circles become straight lines) of a sphere of radius R centred on a point.

$$x = \frac{R\cos\varphi\sin\Delta\lambda}{\cos c}, \quad y = \frac{R\left(\cos\varphi_0\sin\varphi - \sin\varphi_0\cos\varphi\cos\Delta\lambda\right)}{\cos c}$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`lat0`	in	latitude of the centre (decimal degrees)	—
`lon0`	in	longitude of the centre (decimal degrees)	—
`R`	in	radius of the sphere	m
`x`	out	x	m
`y`	out	y	m

Applies when: the point within about 84 degrees of the centre (cos c >= 0.1).

```
$ geospatial-analytics solve geo_gnomonic_forward lat=40 lon=10 lat0=45 lon0=0 "R=6371000 m"
x = 857807.9193 m
y = -508965.7799 m
  by: Gnomonic projection (sphere)  (the orthographic numerators divided by cos c, the cosine of
the angular distance from the centre)
  lat = 40   []
  lon = 10   []
  lat0 = 45   []
  lon0 = 0   []
  R = 6371000 m   []
  assumes: the point within about 84 degrees of the centre (cos c >= 0.1)
```

What the worked example means: This calculation determines the planar coordinates of a specific geographic location using the gnomonic projection method. By taking the input latitude, longitude, and centre coordinates along with the sphere radius R, it computes the x and y positions where great circles appear as straight lines. The resulting values 857808 m and -508966 m represent the distance from the projection origin in meters, valid only if the point is within approximately eighty-four degrees of the centre.

Inverse Albers equal-area conic projection — `geo_albers_equal_area_inverse`

Latitude and longitude of a point given by its easting and northing in the ellipsoidal Albers equal-area conic projection with two standard parallels.

$$q = \frac{C - \rho^2 n^2 / a^2}{n}, \quad \varphi : \; q(\varphi) = q, \quad \lambda = \lambda_0 + \frac{\theta}{n}$$

symbol		meaning	SI unit
`x`	in	easting	m
`y`	in	northing	m
`lat1`	in	first standard parallel (decimal degrees)	—
`lat2`	in	second standard parallel (decimal degrees)	—
`lat0`	in	latitude of origin (decimal degrees)	—
`lon0`	in	central meridian (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`FE`	in	false easting (default 0) (optional)	m
`FN`	in	false northing (default 0) (optional)	m
`lat`	out	latitude (degrees)	—
`lon`	out	longitude (degrees)	—

Applies when: parallels within 89 degrees of the equator and not symmetric about it; the point inside the projected area.

```
$ geospatial-analytics solve geo_albers_equal_area_inverse "x=-338796 m" "y=1952260 m" lat1=29.5
lat2=45.5 lat0=23 lon0=-96 ellipsoid=GRS80
lat = 40.51852456
lon = -100.0339515
  by: Inverse Albers equal-area conic projection  (rho and theta from the offsets, q = (C - (rho
n/a)^2)/n, latitude from q by Newton iteration; lambda = lambda0 + theta/n)
  x = -338796 m   []
  y = 1952260 m   []
  lat1 = 29.5   []
  lat2 = 45.5   []
  lat0 = 23   []
  lon0 = -96   []
  ellipsoid = GRS80
  assumes: parallels within 89 degrees of the equator and not symmetric about it; the point inside
the projected area
```

What the worked example means: This calculation determines the geographic coordinates corresponding to the projected easting of -338796 m and northing of 1.95226e+06 m. Using the specified standard parallels at 29.5 and 45.5, along with the origin latitude 23 and central meridian -96, the algorithm computes the resulting latitude as 40.5185 and longitude as -100.034. These values represent the original location on the reference ellipsoid before it was transformed into the planar map system.

Inverse Lambert conformal conic projection (two standard parallels) — `geo_lambert_conformal_conic_inverse`

Latitude and longitude of a point given by its easting and northing in the ellipsoidal Lambert conformal conic projection with two standard parallels.

$$\rho = \operatorname{sgn}(n)\sqrt{x'^2 + (\rho_0 - y')^2}, \quad t = \left(\frac{\rho}{aF}\right)^{1/n}, \quad \lambda = \lambda_0 + \frac{\theta}{n}$$

symbol		meaning	SI unit
`x`	in	easting	m
`y`	in	northing	m
`lat1`	in	first standard parallel (decimal degrees)	—
`lat2`	in	second standard parallel (decimal degrees)	—
`lat0`	in	latitude of origin (decimal degrees)	—
`lon0`	in	central meridian (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`FE`	in	false easting (default 0) (optional)	m
`FN`	in	false northing (default 0) (optional)	m
`lat`	out	latitude (degrees)	—
`lon`	out	longitude (degrees)	—

Applies when: parallels within 89 degrees of the equator and not symmetric about it.

```
$ geospatial-analytics solve geo_lambert_conformal_conic_inverse "x=-339545 m" "y=1937380 m"
lat1=33 lat2=45 lat0=23 lon0=-96 ellipsoid=GRS80
lat = 40.29658075
lon = -100.01589
  by: Inverse Lambert conformal conic projection (two standard parallels)  (rho and theta from the
offsets, t = (rho/(aF))^(1/n), isometric latitude -ln t, geodetic latitude by Newton iteration;
lambda = lambda0 + theta/n)
  x = -339545 m   []
  y = 1937380 m   []
  lat1 = 33   []
  lat2 = 45   []
  lat0 = 23   []
  lon0 = -96   []
  ellipsoid = GRS80
  assumes: parallels within 89 degrees of the equator and not symmetric about it
```

What the worked example means: This calculation converts the provided easting of -339545 m and northing of 1.93738e+06 m back into geographic coordinates using the ellipsoidal Lambert conformal conic projection with two standard parallels. By reversing the forward transformation, the algorithm determines the isometric latitude through iterative Newton methods and computes the final longitude relative to the central meridian of -96. The resulting latitude of 40.2966 and longitude of -100.016 represent the precise global position corresponding to the projected map coordinates, assuming the reference ellipsoid and false origins are correctly defined.

Inverse Mercator projection (ellipsoidal) — `geo_mercator_inverse`

Latitude and longitude of a point given by its x and y in the ellipsoidal Mercator projection with a central meridian and standard parallel.

$$\psi = \frac{y}{k_0 a}, \quad \tau' = \sinh\psi, \quad \tau = \tan\varphi : \ \tau\sqrt{1+\sigma^2} - \sigma\sqrt{1+\tau^2} = \tau'$$

symbol		meaning	SI unit
`x`	in	easting	m
`y`	in	northing	m
`lon0`	in	central meridian (decimal degrees)	—
`lat_ts`	in	latitude of true scale (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`lat`	out	latitude (degrees)	—
`lon`	out	longitude (degrees)	—

Applies when: |lat_ts| <= 85.

```
$ geospatial-analytics solve geo_mercator_inverse "x=-14471.5 m" "y=6680752 m" lon0=0 lat_ts=0
ellipsoid=WGS84
lat = 51.5223892
lon = -0.1299996963
  by: Inverse Mercator projection (ellipsoidal)  (isometric latitude psi = y/(k0 a); conformal
tangent sinh(psi); geodetic latitude by Newton iteration (Karney's tau method))
  x = -14471.5 m   []
  y = 6680752 m   []
  lon0 = 0   []
  lat_ts = 0   []
  ellipsoid = WGS84
  assumes: |lat_ts| <= 85
```

What the worked example means: This calculation determines the geographic coordinates from planar values. The input easting is -14471.5 m meters and northing is 6.68075e+06 m meters, referenced to a central meridian of 0 degrees and a standard parallel of 0 degrees. Using an ellipsoidal Mercator projection with a scale factor of one, the method applies Newton iteration on the conformal tangent to recover the geodetic latitude. The resulting position is located at latitude 51.5224 and longitude -0.13, providing the global reference for the local grid point.

Lambert azimuthal equal-area projection (ellipsoidal) — `geo_lambert_azimuthal_equal_area_forward`

Easting and northing of a point in the ellipsoidal oblique Lambert azimuthal equal-area projection about a centre (as in ETRS89-LAEA, EPSG:3035).

$$B = R_q\sqrt{\frac{2}{1+\sin\beta_1\sin\beta+\cos\beta_1\cos\beta\cos\Delta\lambda}},\ x = BD\cos\beta\sin\Delta\lambda,\ y = \frac{B}{D}(\cos\beta_1\sin\beta-\sin\beta_1\cos$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`lat0`	in	latitude of the centre (decimal degrees)	—
`lon0`	in	longitude of the centre (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`FE`	in	false easting (default 0) (optional)	m
`FN`	in	false northing (default 0) (optional)	m
`x`	out	easting	m
`y`	out	northing	m

Applies when: centre within 89 degrees of the equator; point within 120 degrees of the centre.

```
$ geospatial-analytics solve geo_lambert_azimuthal_equal_area_forward lat=50 lon=5 lat0=52 lon0=10
ellipsoid=GRS80 "FE=4321000 m" "FN=3210000 m"
x = 3962799.451 m
y = 2999718.853 m
  by: Lambert azimuthal equal-area projection (ellipsoidal)  (authalic latitudes beta = asin(q/q_p)
on the authalic sphere R_q, then the spherical azimuthal equal-area formulas with Snyder's D
factor)
  lat = 50   []
  lon = 5   []
  lat0 = 52   []
  lon0 = 10   []
  FE = 4321000 m   []
  FN = 3210000 m   []
  ellipsoid = GRS80
  assumes: centre within 89 degrees of the equator; point within 120 degrees of the centre
```

What the worked example means: This calculation determines the map coordinates for a specific location using the ellipsoidal Lambert azimuthal equal-area projection. By converting geographic latitude and longitude into planar distances, it preserves area relationships across the map surface. The resulting easting of 3.9628e+06 m and northing of 2.99972e+06 m indicate how far the point lies from the defined false origin in meters, allowing for accurate spatial analysis and thematic mapping where proportional area representation is essential.

Lambert conformal conic projection (two standard parallels): coordinates, scale factor and convergence — `geo_lambert_conformal_conic_forward`

Easting and northing of a point in the ellipsoidal Lambert conformal conic projection with two standard parallels (EPSG method 9802), with the point scale factor and the grid convergence.

$$n = \frac{\ln m_1 - \ln m_2}{\ln t_1 - \ln t_2},\ \rho = aFt^n,\ x = FE + \rho\sin\theta,\ y = FN + \rho_0 - \rho\cos\theta,\ k = \frac{\rho n}{am}$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`lat1`	in	first standard parallel (decimal degrees)	—
`lat2`	in	second standard parallel (decimal degrees)	—
`lat0`	in	latitude of origin (decimal degrees)	—
`lon0`	in	central meridian (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`FE`	in	false easting (default 0) (optional)	m
`FN`	in	false northing (default 0) (optional)	m
`x`	out	easting	m
`y`	out	northing	m
`k`	out	point scale factor	—
`gamma`	out	grid convergence (degrees)	—

Applies when: parallels within 89 degrees of the equator and not symmetric about it.

```
$ geospatial-analytics solve geo_lambert_conformal_conic_forward lat=40 lon=-100 lat1=33 lat2=45
lat0=23 lon0=-96 ellipsoid=GRS80
x = -339643.7785 m
y = 1904595.046 m
k = 0.9946660436
gamma = -2.521985006
  by: Lambert conformal conic projection (two standard parallels): coordinates, scale factor and
convergence  (n = (ln m1 - ln m2)/(ln t1 - ln t2), F = m1/(n t1^n), rho = a F t^n with t = exp(-
psi); x = FE + rho sin(theta), y = FN + rho0 - rho cos(theta), theta = n (lambda - lambda0); k =
rho n/(a m); convergence = theta)
  lat = 40   []
  lon = -100   []
  lat1 = 33   []
  lat2 = 45   []
  lat0 = 23   []
  lon0 = -96   []
  ellipsoid = GRS80
  assumes: parallels within 89 degrees of the equator and not symmetric about it
```

What the worked example means: This calculation determines the Lambert conformal conic coordinates for a specific location using two standard parallels. The resulting easting of -339644 m and northing of 1.9046e+06 m define the point's position on the projected grid, while the scale factor of 0.994666 indicates how much distances are distorted at that spot compared to the true ellipsoid surface. The grid convergence value of -2.52199 represents the angular difference between true north and grid north at this location.

Lambert cylindrical equal-area projection (sphere) — `geo_cylindrical_equal_area_forward`

x and y of a point in the cylindrical equal-area projection of a sphere of radius R with a standard parallel (0 Lambert, 30 Behrmann, 45 Gall-Peters).

$$x = R(\lambda - \lambda_0)\cos\varphi_s, \quad y = \frac{R\sin\varphi}{\cos\varphi_s}$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`lat_ts`	in	standard parallel (decimal degrees)	—
`lon0`	in	central meridian (decimal degrees)	—
`R`	in	radius of the sphere	m
`x`	out	x	m
`y`	out	y	m

Applies when: standard parallel strictly between -90 and 90.

```
$ geospatial-analytics solve geo_cylindrical_equal_area_forward lat=45 lon=10 lat_ts=45 lon0=0
"R=6371000 m"
x = 786266.8666 m
y = 6371000 m
  by: Lambert cylindrical equal-area projection (sphere)  (x = R (lambda - lambda0) cos(lat_ts), y
= R sin(phi)/cos(lat_ts))
  lat = 45   []
  lon = 10   []
  lat_ts = 45   []
  lon0 = 0   []
  R = 6371000 m   []
  assumes: standard parallel strictly between -90 and 90
```

What the worked example means: This calculation determines the planar Cartesian coordinates for a specific location using the Lambert cylindrical equal-area projection on a sphere. By taking the latitude of 45, the longitude of 10, the standard parallel of 45, the central meridian of 0, and the sphere radius of 6.371e+06 m, it produces an east-west position x of 786267 m and a north-south position y of 6.371e+06 m. This method preserves area proportions, making it suitable for thematic maps where statistical accuracy regarding size is more important than shape fidelity.

Latitude and longitude to UTM coordinates (with scale factor and convergence) — `geo_utm_forward`

UTM zone, hemisphere, easting and northing of a point on the WGS84 or GRS80 ellipsoid, with the point scale factor and grid convergence.

$$E = 500\,000 + 0.9996\,A\eta, \quad N = N_0 + 0.9996\,A\xi$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`easting`	out	UTM easting	m
`northing`	out	UTM northing	m
`zone`	out	UTM zone number	—
`hemisphere`	out	N or S	—
`k`	out	point scale factor	—
`gamma`	out	grid convergence (degrees)	—

Applies when: 80 S <= latitude <= 84 N.

```
$ geospatial-analytics solve geo_utm_forward lat=51.5 lon=-0.13 ellipsoid=WGS84
easting = 699195.8759 m
northing = 5709335.019 m
k = 1.000087189
gamma = 2.246819165
zone = 30
hemisphere = N
  by: Latitude and longitude to UTM coordinates (with scale factor and convergence)  (zone from the
longitude (Norway and Svalbard exceptions), then transverse Mercator by Krueger's series with k0 =
0.9996, false easting 500 km, false northing 10000 km in the south)
  lat = 51.5   []
  lon = -0.13   []
  ellipsoid = WGS84
  assumes: 80 S <= latitude <= 84 N
```

What the worked example means: This calculation converts geographic coordinates into the Universal Transverse Mercator system. Using the input latitude of 51.5 and longitude of -0.13, it identifies the correct zone as 30 in the N hemisphere. The resulting projected position is given by an easting of 699196 m and a northing of 5.70934e+06 m. Additionally, the local scale factor is computed as 1.00009 and the grid convergence as 2.24682, which describe how distances and directions are distorted on the flat map relative to the curved earth.

Latitude and longitude to Web Mercator (EPSG:3857) — `geo_web_mercator_forward`

Web Mercator (pseudo-Mercator, EPSG:3857) x and y of a WGS84 latitude and longitude, as used by web maps.

$$x = R\lambda, \quad y = R \ln \tan\left(\frac{\pi}{4} + \frac{\varphi}{2}\right), \quad R = 6378137\ \mathrm{m}$$

symbol		meaning	SI unit
`lat`	in	latitude (WGS84) (decimal degrees)	—
`lon`	in	longitude (WGS84) (decimal degrees)	—
`x`	out	Web Mercator x	m
`y`	out	Web Mercator y	m

Applies when: |latitude| <= 85.06 degrees (the square web-map extent).

```
$ geospatial-analytics solve geo_web_mercator_forward lat=51.5 lon=-0.13
x = -14471.5338 m
y = 6710219.083 m
  by: Latitude and longitude to Web Mercator (EPSG:3857)  (spherical Mercator formulas on a sphere
of radius 6378137 m applied to WGS84 coordinates)
  lat = 51.5   []
  lon = -0.13   []
  assumes: |latitude| <= 85.06 degrees (the square web-map extent)
```

What the worked example means: This calculation converts the provided WGS84 latitude of 51.5 and longitude of -0.13 into Web Mercator projected coordinates. The resulting easting is -14471.5 m and the

northing is 6.71022e+06 m, which represent the position on a spherical map projection commonly used by online mapping services.

Mercator projection (ellipsoidal): coordinates and scale factor — `geo_mercator_forward`

x and y of a point in the ellipsoidal Mercator projection with a central meridian and standard parallel (lat_ts = 0 is World Mercator, EPSG:3395), with the point scale factor.

$$x = k_0 a(\lambda - \lambda_0), \quad y = k_0 a\left[\operatorname{arsinh}(\tan\varphi) - e\operatorname{artanh}(e\sin\varphi)\right], \quad k = \frac{k_0\sqrt{1 - e^2\sin^2\varphi}}{\cos\varphi}$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`lon0`	in	central meridian (decimal degrees)	—
`lat_ts`	in	latitude of true scale (standard parallel) (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`x`	out	easting	m
`y`	out	northing	m
`k`	out	point scale factor	—

Applies when: |latitude| <= 89.5; |lat_ts| <= 85.

```
$ geospatial-analytics solve geo_mercator_forward lat=51.5 lon=-0.13 lon0=0 lat_ts=0
ellipsoid=WGS84
x = -14471.5338 m
y = 6676757.754 m
k = 1.603091339
  by: Mercator projection (ellipsoidal): coordinates and scale factor  (x = k0 a (lambda -
lambda0), y = k0 a psi with the isometric latitude psi; k0 = cos(lat_ts)/sqrt(1 - e^2 sin^2
lat_ts))
  lat = 51.5   []
  lon = -0.13   []
  lon0 = 0   []
  lat_ts = 0   []
  ellipsoid = WGS84
  assumes: |latitude| <= 89.5; |lat_ts| <= 85
```

What the worked example means: This calculation determines the projected coordinates for a specific geographic location using the ellipsoidal Mercator projection, which accounts for the Earth's flattening rather than assuming a perfect sphere. The resulting easting of -14471.5 m and northing of 6.67676e+06 m represent the position on the plane relative to the central meridian and standard parallel. Additionally, the point scale factor of 1.60309 indicates how much distances are distorted at this particular latitude compared to the true scale line.

Mollweide equal-area projection (sphere) — `geo_mollweide_forward`

x and y of a point in the Mollweide (homalographic) equal-area projection of a sphere of radius R.

$$2\theta + \sin 2\theta = \pi \sin\varphi, \quad x = \frac{2\sqrt{2}}{\pi} R\,\Delta\lambda \cos\theta, \quad y = \sqrt{2} R \sin\theta$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`lon0`	in	central meridian (decimal degrees)	—
`R`	in	radius of the sphere	m
`x`	out	x	m
`y`	out	y	m

Applies when: sphere of radius R > 0.

```
$ geospatial-analytics solve geo_mollweide_forward lat=45 lon=90 lon0=0 "R=6371000 m"
x = 7261188.135 m
y = 5334269.291 m
  by: Mollweide equal-area projection (sphere)  (auxiliary angle theta from 2 theta + sin 2 theta =
pi sin(phi) by Newton's method; x = (2 sqrt2/pi) R dlambda cos(theta), y = sqrt2 R sin(theta))
  lat = 45   []
  lon = 90   []
  lon0 = 0   []
  R = 6371000 m   []
  assumes: sphere of radius R > 0
```

What the worked example means: This calculation determines the planar x and y coordinates for a specific geographic location using the Mollweide projection. It takes the latitude, longitude, central meridian, and sphere radius as inputs to produce 7.26119e+06 m for the horizontal axis and 5.33427e+06 m for the vertical axis. The result places the point within an elliptical frame that preserves area, meaning landmasses retain their correct relative sizes compared to each other, although shapes may be distorted near the edges.

Orthographic projection (sphere) — `geo_orthographic_forward`

x and y of a point in the orthographic projection (the view of a globe from infinitely far away) of a sphere of radius R centred on a point.

$$x = R\cos\varphi \sin\Delta\lambda, \quad y = R\left(\cos\varphi_0 \sin\varphi - \sin\varphi_0 \cos\varphi \cos\Delta\lambda\right)$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`lat0`	in	latitude of the centre (decimal degrees)	—
`lon0`	in	longitude of the centre (decimal degrees)	—
`R`	in	radius of the sphere	m
`x`	out	x	m
`y`	out	y	m

Applies when: the point on the visible hemisphere (cos c >= 0).

```
$ geospatial-analytics solve geo_orthographic_forward lat=40 lon=10 lat0=45 lon0=0 "R=6371000 m"
x = 847484.5736 m
y = -502840.5978 m
  by: Orthographic projection (sphere)  (x = R cos(phi) sin(dlambda), y = R (cos(phi0) sin(phi) -
sin(phi0) cos(phi) cos(dlambda)))
  lat = 40   []
  lon = 10   []
  lat0 = 45   []
  lon0 = 0   []
  R = 6371000 m   []
  assumes: the point on the visible hemisphere (cos c >= 0)
```

What the worked example means: This calculation determines the two-dimensional position of a specific geographic location within an orthographic projection, which simulates viewing the Earth from an infinite distance. Using the input latitude 40 and longitude 10, along with the center point defined by latitude 45 and longitude 0, the algorithm computes the horizontal coordinate x as 847485 m and the vertical coordinate y as -502841 m. These values represent the projected location on a spherical surface with radius 6.371e+06 m, effectively mapping the curved globe onto a flat plane while preserving the appearance of a hemisphere.

Polar stereographic projection (standard parallel): coordinates and scale factor — `geo_polar_stereographic_forward`

Easting and northing of a point in the ellipsoidal polar stereographic projection defined by a latitude of true scale (EPSG variant B, as in EPSG:3413 and EPSG:3031), with the point scale factor.

$$\rho = a\, m_c \frac{t}{t_c}, \quad t = e^{-\psi}, \quad x = FE + \rho \sin \Delta\lambda,\; y = FN \mp \rho \cos \Delta\lambda, \quad k = \frac{\rho}{am}$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`pole`	in	pole at the projection centre	—
`lat_ts`	in	latitude of true scale (decimal degrees)	—
`lon0`	in	longitude of the downward (north pole) or upward (south pole) y axis (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`FE`	in	false easting (default 0) (optional)	m
`FN`	in	false northing (default 0) (optional)	m
`x`	out	easting	m
`y`	out	northing	m
`k`	out	point scale factor	—

Applies when: the point in the pole's hemisphere (not at the pole); 30 <= |lat_ts| <= 89.99 in the same hemisphere.

```
$ geospatial-analytics solve geo_polar_stereographic_forward lat=75 lon=-40 pole=north lat_ts=70
lon0=-45 ellipsoid=WGS84
x = 142401.9812 m
y = -1627662.093 m
k = 0.986664282
  by: Polar stereographic projection (standard parallel): coordinates and scale factor  (rho = a
m_c t/t_c with t = exp(-psi); x = FE + rho sin(dlambda), y = FN -/+ rho cos(dlambda); k = rho/(a
m))
  lat = 75   []
  lon = -40   []
  lat_ts = 70   []
  lon0 = -45   []
  pole = north
  ellipsoid = WGS84
  assumes: the point in the pole's hemisphere (not at the pole); 30 <= |lat_ts| <= 89.99 in the
same hemisphere
```

What the worked example means: This calculation determines the planar coordinates for a specific location on Earth using an ellipsoidal polar stereographic projection. By processing the input latitude of 75 degrees, longitude of -40 degrees, and the defined true scale latitude of 70 degrees, the model derives the easting value of 142402 m meters and the northing value of -1.62766e+06 m meters. Additionally, it computes the local distortion metric, known as the point scale factor, which equals 0.986664. These results allow precise mapping in polar regions where traditional projections fail to maintain accurate area or shape relationships.

Sinusoidal equal-area projection (ellipsoidal) — `geo_sinusoidal_forward`

x and y of a point in the ellipsoidal sinusoidal (Sanson-Flamsteed) equal-area projection, as used for MODIS land grids.

$$x = \frac{a\cos\varphi}{\sqrt{1 - e^2\sin^2\varphi}}(\lambda - \lambda_0), \quad y = m(\varphi)$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`lon0`	in	central meridian (decimal degrees)	—
`ellipsoid`	in	reference ellipsoid	—
`x`	out	easting	m
`y`	out	northing	m

Applies when: |latitude| <= 89.9.

```
$ geospatial-analytics solve geo_sinusoidal_forward lat=45 lon=10 lon0=0 ellipsoid=WGS84
x = 788468.3509 m
y = 4984944.378 m
  by: Sinusoidal equal-area projection (ellipsoidal)  (x = N cos(phi) (lambda - lambda0), y =
meridian distance m(phi) (Helmert's series))
  lat = 45   []
  lon = 10   []
  lon0 = 0   []
  ellipsoid = WGS84
  assumes: |latitude| <= 89.9
```

What the worked example means: This calculation determines the planar coordinates for a specific geographic location using the ellipsoidal sinusoidal projection. The easting, reported as 788468 m, depends on the latitude and the difference between the longitude and the central meridian. The northing, reported as 4.98494e+06 m, represents the distance along the central meridian from the equator to the given latitude. This method ensures that area is preserved across the map, which is essential for global datasets.

Tissot's indicatrix: scale factors and distortion of a projection at a point — `geo_tissot_indicatrix`

The meridian and parallel scale factors h and k, the angle between meridian and parallel, the semi-axes a and b of Tissot's indicatrix, the maximum angular distortion and the areal scale of a world projection of the sphere at a point.

$$a' = \sqrt{h^2 + k^2 + 2hk\sin\theta'},\ b' = \sqrt{h^2 + k^2 - 2hk\sin\theta'},\ a = \tfrac{a'+b'}{2},\ b = \tfrac{a'-b'}{2},\ \omega = 2\arcsin\frac{a-b}{a+b},\ s =$$

symbol		meaning	SI unit
`projection`	in	projection (spherical form, central meridian 0)	—
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude measured from the central meridian (decimal degrees)	—
`h`	out	scale factor along the meridian	—
`k`	out	scale factor along the parallel	—
`theta_prime`	out	angle between the projected meridian and parallel (degrees, at most 90)	—
`a`	out	maximum scale factor (semi-major axis)	—
`b`	out	minimum scale factor (semi-minor axis)	—
`omega`	out	maximum angular distortion (degrees)	—
`s`	out	areal scale factor	—

Applies when: |latitude| <= 80.

```
$ geospatial-analytics solve geo_tissot_indicatrix projection=mercator lat=60 lon=0
h = 2
k = 2
theta_prime = 90
a = 2
b = 2
omega = 0
s = 4
  by: Tissot's indicatrix: scale factors and distortion of a projection at a point  (partial
derivatives of the projection: h = |d(x,y)/dphi|, k = |d(x,y)/dlambda|/cos(phi), sin(theta') from
their cross product; a' = sqrt(h^2+k^2+2hk sin theta'), b' = sqrt(h^2+k^2-2hk sin theta'), a =
(a'+b')/2, b = (a'-b')/2, omega = 2 asin((a-b)/(a+b)), s = hk sin theta')
  lat = 60   []
  lon = 0   []
  projection = mercator
  assumes: |latitude| <= 80
```

What the worked example means: The model computes local distortion metrics for a spherical projection at the specified location. Using partial derivatives, it determines the meridian scale factor 2 and parallel scale factor 2. From these, it derives the angle between projected meridians and parallels as 90 degrees, the semi-major axis of Tissot's indicatrix as 2, the semi-minor axis as 2, the maximum angular distortion as 0 degrees, and the areal scale factor as 4. These values quantify how lengths, angles, and areas deviate from their true globe proportions at this point.

Transverse Mercator projection: coordinates, scale factor and grid convergence — `geo_transverse_mercator_forward`

Easting and northing of a point in a transverse Mercator projection with a given origin, central-meridian scale factor and false origin on the WGS84 or GRS80 ellipsoid, with the point scale factor and the grid convergence.

$$\xi = \xi' + \sum_{j=1}^{6} \alpha_j \sin 2j\xi' \cosh 2j\eta', \quad \eta = \eta' + \sum_{j=1}^{6} \alpha_j \cos 2j\xi' \sinh 2j\eta', \quad x = FE + k_0 A\eta,\ y = FN + k_0$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`lat0`	in	latitude of origin (decimal degrees)	—
`lon0`	in	central meridian (decimal degrees)	—
`k0`	in	scale factor on the central meridian	—
`ellipsoid`	in	reference ellipsoid	—
`FE`	in	false easting (default 0) (optional)	m
`FN`	in	false northing (default 0) (optional)	m
`x`	out	easting	m
`y`	out	northing	m
`k`	out	point scale factor	—
`gamma`	out	grid convergence (degrees, positive east of the central meridian in the north)	—

Applies when: within 10 degrees of longitude of the central meridian; latitude within 89 degrees of the equator.

```
$ geospatial-analytics solve geo_transverse_mercator_forward lat=52.5 lon=13.4 lat0=0 lon0=15
k0=0.9996 ellipsoid=GRS80 "FE=500000 m" "FN=0 m"
x = 391390.7313 m
y = 5817855.241 m
k = 0.9997447935
gamma = -1.269488544
  by: Transverse Mercator projection: coordinates, scale factor and grid convergence  (Krueger's
series to sixth order in the third flattening (conformal latitude, then the Gauss-Krueger sums);
scale and convergence from the series' derivatives (Karney 2011))
  lat = 52.5   []
  lon = 13.4   []
  lat0 = 0   []
  lon0 = 15   []
  k0 = 0.9996   []
  FE = 500000 m   []
  FN = 0 m   []
  ellipsoid = GRS80
  assumes: within 10 degrees of longitude of the central meridian; latitude within 89 degrees of
the equator
```

What the worked example means: This calculation converts the geographic coordinates of latitude 52.5 and longitude 13.4 into projected plane coordinates using the Transverse Mercator method on the GRS80 ellipsoid. The resulting easting is 391391 m meters and the northing is 5.81786e+06 m meters, relative to a false origin defined by false easting 500000 m and false northing 0 m. Additionally, the projection computes the local distortion metrics: the point scale factor is 0.999745, indicating how much distances are stretched at this location compared to the central meridian, and the grid convergence is -1.26949, which represents the angle between true north and the vertical grid line.

UTM coordinates to latitude and longitude — `geo_utm_inverse`

Latitude and longitude on the WGS84 or GRS80 ellipsoid of a point given by its UTM easting, northing, zone and hemisphere.

$$\xi' = \xi - \sum_{j=1}^{6} \beta_j \sin 2j\xi \cosh 2j\eta, \quad \eta' = \eta - \sum_{j=1}^{6} \beta_j \cos 2j\xi \sinh 2j\eta$$

symbol		meaning	SI unit
`easting`	in	UTM easting	m
`northing`	in	UTM northing	m
`zone`	in	UTM zone number	—
`hemisphere`	in	hemisphere of the zone (N: false northing 0; S: false northing 10000 km)	—
`ellipsoid`	in	reference ellipsoid	—
`lat`	out	latitude (degrees)	—
`lon`	out	longitude (degrees)	—

Applies when: 100 km <= easting <= 900 km; 0 <= northing <= 10000 km.

```
$ geospatial-analytics solve geo_utm_inverse "easting=699316 m" "northing=5710164 m" zone=30
hemisphere=N ellipsoid=WGS84
lat = 51.50740226
lon = -0.1278032352
  by: UTM coordinates to latitude and longitude  (inverse Krueger series to the conformal latitude,
then Newton iteration for the geodetic latitude)
  easting = 699316 m   []
  northing = 5710164 m   []
  ellipsoid = WGS84
  hemisphere = N
  assumes: 100 km <= easting <= 900 km; 0 <= northing <= 10000 km
```

What the worked example means: This calculation converts a point defined by its UTM easting 699316 m and northing 5.71016e+06 m in zone 30 of the N hemisphere into geographic coordinates. The process uses an inverse Krueger series followed by Newton iteration to determine the geodetic position on the WGS84 ellipsoid. The resulting latitude is 51.5074 and the longitude is -0.127803. These values represent the precise location on the Earth's surface corresponding to the provided grid coordinates, assuming they fall within valid UTM bounds.

Web Mercator (EPSG:3857) to latitude and longitude — `geo_web_mercator_inverse`

WGS84 latitude and longitude of a point given by its Web Mercator (pseudo-Mercator) x and y.

$$\varphi = \arctan\sinh\frac{y}{R}, \quad \lambda = \frac{x}{R}$$

symbol		meaning	SI unit
`x`	in	Web Mercator x	m
`y`	in	Web Mercator y	m
`lat`	out	latitude (degrees)	—
`lon`	out	longitude (degrees)	—

Applies when: inside the web-map square.

```
$ geospatial-analytics solve geo_web_mercator_inverse "x=-14471.5 m" "y=6710219.1 m"
lat = 51.50000009
lon = -0.1299996963
  by: Web Mercator (EPSG:3857) to latitude and longitude  (inverse spherical Mercator: lat =
atan(sinh(y/R)), lon = x/R)
  x = -14471.5 m   []
  y = 6710219.1 m   []
  assumes: inside the web-map square
```

What the worked example means: This calculation converts the Web Mercator planar coordinates into geographic latitude and longitude. By applying the inverse spherical Mercator formulas, the input x and y values in meters are transformed into angular degrees. The resulting latitude is 51.5 and the longitude is -0.13, representing the specific location on the Earth's surface corresponding to the given projected coordinates.

Web-map tile (slippy map / XYZ) and quadkey containing a point — `geo_web_map_tile`

The x and y indices of the web-map (XYZ / slippy map) tile containing a WGS84 point at a zoom level, and the tile's quadkey.

$$x_t = \left\lfloor 2^z \, \frac{\lambda + 180^\circ}{360^\circ} \right\rfloor, \quad y_t = \left\lfloor 2^z \left(\frac{1}{2} - \frac{\operatorname{arsinh}(\tan \varphi)}{2\pi} \right) \right\rfloor$$

symbol		meaning	SI unit
`lat`	in	latitude (decimal degrees)	—
`lon`	in	longitude (decimal degrees)	—
`zoom`	in	zoom level (0-24)	—
`tile_x`	out	tile column (from 180 W)	—
`tile_y`	out	tile row (from the north edge)	—
`quadkey`	out	quadkey of the tile	—

Applies when: |latitude| <= 85.0511; zoom 0-24.

```
$ geospatial-analytics solve geo_web_map_tile lat=51.5 lon=-0.13 zoom=12
tile_x = 2046
tile_y = 1362
quadkey = 031313131130
  by: Web-map tile (slippy map / XYZ) and quadkey containing a point  (Web Mercator fractions x =
(lon + 180)/360, y = 1/2 - asinh(tan lat)/(2 pi), scaled by 2^zoom and floored; quadkey digits
interleave the bits of x and y)
  lat = 51.5   []
  lon = -0.13   []
  assumes: |latitude| <= 85.0511; zoom 0-24
```

What the worked example means: This calculation determines the specific web-map tile containing a geographic location. Using the input latitude of 51.5 and longitude of -0.13 at zoom level 12, it converts these coordinates into integer grid indices. The resulting tile column is 2046 and the tile row is 1362. These values are then interleaved to form the unique identifier known as the quadkey, which is 031313131130. This system allows map applications to efficiently retrieve and display the correct image segment for any given location.

Network accessibility (6)

Gravity-based (Hansen) accessibility on a road network — `geo_gravity_accessibility`

The Hansen gravity accessibility of every node of a network: the sum of the opportunities (jobs, services) at the other nodes discounted by a power or negative-exponential function of the shortest-path distance.

$$A_i = \sum_{j \neq i} O_j \, f(d_{ij}), \quad f(d) = d^{-\beta} \text{ or } e^{-\beta d}$$

symbol		meaning	SI unit
`edges`	in	undirected network as weighted edges between nodes numbered 0 .. n-1	—
`opportunities`	in	opportunities at nodes 0 .. n-1	—
`beta`	in	distance-decay parameter	—

symbol		meaning	SI unit
`impedance`	in	impedance function d^-beta or exp(-beta d)	—
`accessibility`	out	accessibility of each node	—

Applies when: positive edge lengths; a node's own opportunities are excluded; unreachable nodes contribute 0.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the Hansen gravity accessibility for each node in the road network. It computes the shortest path distance from every origin to all other nodes using Dijkstra's algorithm, then sums the opportunities at destination nodes discounted by an impedance function based on that distance and the decay parameter beta. The resulting value, [44.3333, 81.6667, 56.6667, 109.167], represents the total accessible potential from a given location, where closer destinations with higher opportunity counts contribute more significantly to the score than distant or sparse ones.

Network connectivity indices: cyclomatic number, alpha, beta and gamma — `geo_network_connectivity_indices`

The graph-theoretic connectivity measures of a (planar) transport network: vertices, edges, components, cyclomatic number and Kansky's alpha, beta and gamma indices.

$$\mu = e - v + p, \quad \alpha = \frac{e - v + p}{2v - 5}, \quad \beta = \frac{e}{v}, \quad \gamma = \frac{e}{3(v - 2)}$$

symbol		meaning	SI unit
`edges`	in	network edges between nodes numbered from 0 (lengths, if given, are ignored)	—
`vertices`	out	number of vertices v	—
`edges`	out	number of distinct edges e	—
`components`	out	connected components p	—
`cyclomatic`	out	cyclomatic number e - v + p	—
`alpha`	out	(e - v + p)/(2v - 5)	—
`beta`	out	e / v	—
`gamma`	out	e / (3(v - 2))	—

Applies when: at least 3 vertices; planar formulas for alpha and gamma.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the structural connectivity of the transport network by counting distinct nodes and links. The result shows there are 4 vertices and 5 edges distributed across 1 connected components. These values allow for the computation of Kansky's indices, specifically an alpha index of 0.666667, a beta index of 1.25, and a gamma index of 0.833333, which together describe the network's complexity and loopiness.

Network detour index (circuity) between two nodes — `geo_detour_index`

The shortest network distance between two nodes of a street network (edge lengths from the node coordinates), the straight-line distance and their ratio, the detour index or circuity.

$$DI = \frac{d_{net}(o,t)}{\|\mathbf{x}_o - \mathbf{x}_t\|}$$

symbol		meaning	SI unit
`coords`	in	x and y of nodes 0 .. n-1 (n x 2, projected coordinates)	—
`edges`	in	street segments between nodes (lengths are computed from the coordinates)	—
`origin`	in	origin node	—
`destination`	in	destination node	—
`network_distance`	out	shortest network distance	—
`euclidean_distance`	out	straight-line distance	—
`detour_index`	out	network / straight-line distance (>= 1)	—

Applies when: straight edges between the node coordinates; destination reachable.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the circuity between two nodes by comparing the shortest path along the street network to the direct Euclidean distance. The resulting ratio, known as the detour index, indicates how much longer the actual travel route is compared to a theoretical straight line. A value of 1.4 suggests that traveling from the origin to the destination requires covering 7 instead of the direct 5.

Network service area (isochrone) of a facility — `geo_service_area`

The nodes of a road network within a travel-cost cutoff of a facility (the service area or isochrone), their number and the largest distance reached.

$$SA(s,c) = \{v : d(s,v) \le c\}$$

symbol		meaning	SI unit
`edges`	in	undirected network as weighted edges between nodes numbered 0 .. n-1	—
`source`	in	facility node	—
`cutoff`	in	maximum network distance or travel time	—
`reachable`	out	nodes within the cutoff (ascending)	—
`count`	out	number of nodes reached (the facility included)	—
`farthest`	out	largest network distance within the cutoff	—

Applies when: positive edge costs; distance <= cutoff counts as reached.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the network service area by finding all nodes reachable from the facility within the specified cutoff. Starting from the source node, distances are computed

along the edges until the limit is exceeded. The result shows that 4 distinct locations fall within this boundary, and the maximum distance covered by any reachable node is 6. This defines the operational reach of the facility based on current network conditions.

Shimbel accessibility index of every node — `geo_shimbel_index`

The Shimbel (total shortest-path distance) index of every node of a connected network, the network total (dispersion) and the most accessible node (smallest index).

$$A_i = \sum_j d_{ij}, \quad D(G) = \sum_i A_i$$

symbol		meaning	SI unit
`edges`	in	undirected network as weighted edges between nodes numbered 0 .. n-1	—
`shimbel`	out	sum of shortest-path distances from each node	—
`total`	out	network dispersion (sum over all nodes)	—
`most_accessible`	out	node with the smallest Shimbel index (lowest number on ties)	—

Applies when: connected network with nodes 0 .. n-1.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation determines the Shimbel accessibility index, which represents the sum of the shortest path distances from every node to all other nodes in the connected network. The resulting indices are [13, 9, 9, 11], indicating how far each location is from the rest of the system. The node with the lowest value, identified as 1, is considered the most central or accessible point. Additionally, the sum of these individual indices yields the total network dispersion, calculated here as 42, which reflects the overall spread and efficiency of connections within the graph.

Two-step floating catchment area (2SFCA) accessibility — `geo_two_step_floating_catchment`

The 2SFCA spatial accessibility of every node to services (doctors, schools): supply-to-population ratios within each site's catchment, summed over the sites within reach of each node.

$$R_j = \frac{S_j}{\sum_{k:d_{kj}\le d_0} P_k}, \quad A_i = \sum_{j:d_{ij}\le d_0} R_j$$

symbol		meaning	SI unit
`edges`	in	undirected network as weighted edges between nodes numbered 0 .. n-1	—
`population`	in	population at nodes 0 .. n-1	—
`supply_nodes`	in	nodes with a service site	—
`capacity`	in	capacity of each site (e.g. physicians)	—
`threshold`	in	catchment threshold (network distance)	—
`accessibility`	out	accessibility of each node (supply per person)	—
`ratios`	out	supply-to-population ratio of each site	—

Applies when: a catchment includes nodes at network distance <= threshold; a site with no population in reach has ratio 0.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This method computes spatial accessibility by first determining the supply-to-population ratio for each service site, shown here as [0.00217391, 0.002], based on the capacity of [5, 2] and the surrounding population. It then sums these ratios for all sites within the threshold distance of 3 to produce the final accessibility value of [0.00217391, 0.00217391, 0.00417391, 0.002] for each node. This result indicates how many units of service are available per person at that location, considering both local demand and nearby supply.

Planar vector analysis (3)

Discrete Frechet distance between two polylines — `geo_discrete_frechet_distance`

The discrete Frechet ('dog-walking') distance between two polylines (GPS tracks, routes) measured over their vertices.

$$c_{i,j} = \max\left(d(P_i, Q_j),\ \min(c_{i-1,j},\ c_{i-1,j-1},\ c_{i,j-1})\right)$$

symbol		meaning	SI unit
`x1`	in	x coordinates of the first line	—
`y1`	in	y coordinates of the first line	—
`x2`	in	x coordinates of the second line	—
`y2`	in	y coordinates of the second line	—
`distance`	out	discrete Frechet distance (coordinate units)	—

Applies when: at least 2 vertices per line.

```
$ geospatial-analytics solve geo_discrete_frechet_distance "x1=[0, 1, 2]" "y1=[0, 0, 0]" "x2=[0,
2]" "y2=[1, 1]"
distance = 1.414213562
  by: Discrete Frechet distance between two polylines  (dynamic programme (Eiter-Mannila): c(i,j) =
max(d(i,j), min(c(i-1,j), c(i-1,j-1), c(i,j-1))))
  assumes: at least 2 vertices per line
```

What the worked example means: This calculation determines the similarity between two paths using the discrete Frechet distance, often visualized as the minimum leash length required for a dog and owner walking along separate tracks. The result of 1.41421 represents the maximum separation encountered during the optimal simultaneous traversal of the vertices defined by inputs x1, y1, x2, and y2.

Douglas-Peucker line simplification — `geo_douglas_peucker_simplification`

The vertices of a polyline kept by the Ramer-Douglas-Peucker simplification at a distance tolerance (indices from 0).

$$d_{max} = \max_{k} \operatorname{dist}(P_k, \overline{P_i P_j}) > \varepsilon \Rightarrow \text{keep } P_k$$

symbol		meaning	SI unit
`x`	in	x coordinates of the vertices in order	—
`y`	in	y coordinates of the vertices	—
`tolerance`	in	distance tolerance (same unit as the coordinates)	—
`kept`	out	indices of the kept vertices (from 0, in order)	—
`n_kept`	out	number of kept vertices	—

Applies when: distinct vertices; tolerance > 0.

```
$ geospatial-analytics solve geo_douglas_peucker_simplification "x=[0, 1, 2, 3, 4, 5, 6, 7]" "y=[0,
0.1, -0.1, 5, 6, 7, 8.1, 9]" tolerance=1
kept = [0.0,2.0,3.0,7.0]
n_kept = 4
  by: Douglas-Peucker line simplification  (recursive: keep the vertex farthest from the segment
joining the ends of a run if its distance exceeds the tolerance, then split there)
  tolerance = 1   []
  assumes: distinct vertices; tolerance > 0
```

What the worked example means: This calculation applies the Ramer-Douglas-Peucker method to simplify a line by removing vertices that fall within a specified distance from the straight line connecting adjacent kept points. Given an input sequence of coordinates and a positive tolerance value, the process recursively identifies the point farthest from the current segment; if this distance exceeds the tolerance, the point is kept, splitting the line into two new segments for further evaluation. The result indicates that 4 specific vertices were preserved in their original order, effectively reducing the complexity of the geometry while maintaining its essential shape within the bounds defined by the 1 threshold.

Shape compactness indices of a polygon (Polsby-Popper, Schwartzberg, convex hull, Reock) — `geo_polygon_compactness`

Area, perimeter and the standard compactness measures of a polygon in projected coordinates (used for districts and land parcels): Polsby-Popper, Schwartzberg, convex-hull ratio and Reock.

$$PP = \frac{4\pi A}{P^2}, \quad S = \frac{P}{2\sqrt{\pi A}}, \quad C_H = \frac{A}{A_{hull}}, \quad R = \frac{A}{\pi r_{mec}^2}$$

symbol		meaning	SI unit
`x`	in	x coordinates of the vertices in order (not repeating the first)	—
`y`	in	y coordinates of the vertices	—
`area`	out	area (square coordinate units)	—
`perimeter`	out	perimeter (coordinate units)	—
`polsby_popper`	out	4 pi A / P^2	—
`schwartzberg`	out	P / (2 sqrt(pi A))	—
`convex_hull_ratio`	out	area / convex hull area	—
`reock`	out	area / area of the minimum enclosing circle	—

Applies when: a simple polygon with distinct vertices.

```
$ geospatial-analytics solve geo_polygon_compactness "x=[0, 4, 4, 0]" "y=[0, 0, 3, 3]"
area = 12
perimeter = 14
polsby_popper = 0.7693696295
schwartzberg = 1.140072528
convex_hull_ratio = 1
reock = 0.6111549815
  by: Shape compactness indices of a polygon (Polsby-Popper, Schwartzberg, convex hull, Reock)
(shoelace area, perimeter, monotone-chain convex hull, Welzl minimum enclosing circle)
  assumes: a simple polygon with distinct vertices
```

What the worked example means: This analysis computes the geometric compactness of the provided polygon using its vertex coordinates. The area is determined to be 12 and the perimeter is 14. These values feed into standard indices, resulting in a Polsby-Popper score of 0.76937, which measures how closely the shape resembles a circle. A higher score indicates greater compactness, while a lower score suggests a more irregular or elongated boundary.

Point pattern analysis (7)

Kernel density estimate of a point pattern at a location — `geo_kernel_density`

The two-dimensional kernel density (per unit area, integrating to 1) and the intensity (events per unit area) of a point pattern at a location, with a stated kernel and bandwidth.

$$\hat{f}(\mathbf{p}) = \frac{1}{n}\sum_i K_h(\|\mathbf{p}-\mathbf{x}_i\|), \quad K_h^{gauss} = \frac{e^{-u^2/2}}{2\pi h^2},\ K_h^{epa} = \frac{2(1-u^2)}{\pi h^2},\ K_h^{unif} = \frac{1}{\pi h^2}$$

symbol		meaning	SI unit
`x`	in	x coordinates of the events	—
`y`	in	y coordinates of the events	—
`px`	in	x of the location	—
`py`	in	y of the location	—
`bandwidth`	in	bandwidth h (coordinate units)	—
`kernel`	in	kernel (gaussian with sd h; epanechnikov and uniform with radius h)	—
`density`	out	density (per square coordinate unit, integrates to 1)	—
`intensity`	out	intensity = n x density (events per square unit)	—

Applies when: bandwidth > 0.

```
$ geospatial-analytics solve geo_kernel_density "x=[0, 1, 2, 5]" "y=[0, 1, 0, 5]" px=1 py=0.5
bandwidth=2 kernel=gaussian
density = 0.02676479928
intensity = 0.1070591971
  by: Kernel density estimate of a point pattern at a location  (f(p) = (1/n) sum K_h(|p - x_i|)
with 2-D normalized kernels: gaussian exp(-u^2/2)/(2 pi h^2), epanechnikov 2(1 - u^2)/(pi h^2),
uniform 1/(pi h^2) for u = d/h < 1)
  px = 1   []
  py = 0.5   []
  bandwidth = 2   []
  kernel = gaussian
  assumes: bandwidth > 0
```

What the worked example means: This calculation determines the kernel density estimate for a point pattern at a specific location using a bandwidth of 2. The resulting density value of 0.0267648 represents the probability mass per square coordinate unit, while the intensity value of 0.107059 indicates the expected number of events per square unit. These metrics help identify spatial clustering or hotspots by smoothing discrete points into a continuous surface based on their proximity to the target coordinates.

Mean centre and standard distance of a point pattern — `geo_mean_center_standard_distance`

The mean centre of a set of points and the standard distance (root-mean-square distance from the mean centre).

$$SD = \sqrt{\frac{\sum_i (x_i - \bar{x})^2 + (y_i - \bar{y})^2}{n}}$$

symbol		meaning	SI unit
`x`	in	x coordinates	—
`y`	in	y coordinates	—
`mean_x`	out	x of the mean centre	—
`mean_y`	out	y of the mean centre	—
`std_distance`	out	standard distance	—

Applies when: population form (divide by n).

```
$ geospatial-analytics solve geo_mean_center_standard_distance "x=[0, 4, 4, 0]" "y=[0, 0, 3, 3]"
mean_x = 2
mean_y = 1.5
std_distance = 2.5
  by: Mean centre and standard distance of a point pattern  (mean of the coordinates; standard
distance sqrt(sum ((x - xbar)^2 + (y - ybar)^2)/n))
  assumes: population form (divide by n)
```

What the worked example means: This calculation determines the average location of a set of points by computing the mean of their coordinates, resulting in a central point at 2 and 1.5. It also measures how spread out these points are around that centre using the population form of standard distance, which yields a value of 2.5. This provides a simple summary of where features are clustered on average and the typical deviation from that central location.

Nearest-neighbour index (Clark-Evans ratio) of a point pattern — `geo_nearest_neighbour_index`

The mean observed nearest-neighbour distance of a point pattern, the mean expected under complete spatial randomness in a study area, their ratio R (R < 1 clustered, R > 1 dispersed), the z-score and two-sided p-value.

$$R = \frac{\bar{d}_{obs}}{0.5/\sqrt{n/A}}, \quad z = \frac{\bar{d}_{obs} - \bar{d}_{exp}}{0.26136/\sqrt{n^2/A}}$$

symbol		meaning	SI unit
`x`	in	x coordinates of the points	—
`y`	in	y coordinates of the points	—
`area`	in	area of the study region (square coordinate units)	—
`d_obs`	out	mean observed nearest-neighbour distance	—
`d_exp`	out	expected mean distance 0.5/sqrt(n/A)	—
`R`	out	nearest-neighbour index d_obs/d_exp	—
`z`	out	z-score	—
`p_value`	out	two-sided p-value	—

Applies when: distinct points; no edge correction.

```
$ geospatial-analytics solve geo_nearest_neighbour_index "x=[1, 2, 8, 9, 5]" "y=[1, 2, 8, 9, 5]"
area=100
d_obs = 1.979898987
d_exp = 2.236067977
R = 0.8854377448
z = -0.4900692343
p_value = 0.6240849077
  by: Nearest-neighbour index (Clark-Evans ratio) of a point pattern  (mean of each point's
distance to its nearest neighbour; Clark-Evans expectation 0.5/sqrt(density) and standard error
0.26136/sqrt(n^2/A))
  area = 100   []
  assumes: distinct points; no edge correction
```

What the worked example means: This calculation determines the Clark-Evans ratio by comparing the mean observed distance to the nearest neighbour, 1.9799, against the expected distance under complete spatial randomness, 2.23607. The resulting index, 0.885438, quantifies the degree of clustering or dispersion. A value less than one suggests clustering, while a value greater than one indicates dispersion. The statistical significance of this deviation from randomness is assessed using a z-score of -0.490069 and a two-sided p-value of 0.624085.

Quadrat analysis: chi-square test of complete spatial randomness — `geo_quadrat_test`

Counts of points in an nx x ny grid of quadrats over the bounding rectangle of a point pattern, the chi-square statistic against complete spatial randomness, its p-value and the variance-to-mean ratio.

$$\chi^2 = \sum_c \frac{(O_c - E)^2}{E}, \quad E = \frac{n}{n_x n_y}, \quad VMR = \frac{\chi^2}{n_x n_y - 1}$$

symbol		meaning	SI unit
`x`	in	x coordinates of the points	—
`y`	in	y coordinates of the points	—
`nx`	in	number of quadrat columns	—
`ny`	in	number of quadrat rows	—
`chi2`	out	chi-square statistic	—
`df`	out	degrees of freedom (cells - 1)	—
`p_value`	out	upper-tail p-value	—
`vmr`	out	variance-to-mean ratio of the counts (chi2/df)	—
`counts`	out	count of each quadrat, row by row from the south-west	—

Applies when: the window is the bounding rectangle of the points; cells include their west and south edges; the last column and row include the east and north edges.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This analysis divides the bounding rectangle into a grid of equal-sized cells and counts the points in each quadrat. The expected count per cell is derived from the total number of points divided by the number of cells. By comparing observed counts to this uniform expectation, the method calculates a chi-square statistic of 0.4 based on 2 columns and 2 rows. A p-value of 0.940242 indicates the probability that such deviation would occur by chance if the points were randomly distributed. The variance-to-mean ratio of 0.133333 further characterizes the dispersion, where values near one suggest randomness, greater than one indicate clustering, and less than one suggest regularity.

Ripley's K and L functions of a point pattern — `geo_ripley_k`

Ripley's K function and its variance-stabilized L function at given distances for a point pattern, without edge correction, in the bounding rectangle of the points.

$$\hat{K}(d) = \frac{A}{n^2} \sum_{i \neq j} \mathbf{1}(d_{ij} < d), \quad L(d) = \sqrt{K(d)/\pi}$$

symbol		meaning	SI unit
`x`	in	x coordinates of the points	—
`y`	in	y coordinates of the points	—
`d`	in	distances at which to evaluate K	—
`K`	out	K(d) at each distance	—
`L`	out	L(d) = sqrt(K/pi) at each distance	—
`area`	out	area of the bounding rectangle	—

Applies when: window = bounding rectangle of the points; pairs closer than d (strictly) are counted; no edge correction.

```
$ geospatial-analytics solve geo_ripley_k "x=[0, 1, 2, 8, 9, 10]" "y=[0, 1, 0, 9, 10, 9]" "d=[1.5,
3, 12]"
K = [22.2222222222,33.3333333333,44.4444444444]
L = [2.65961520268,3.25735007935,3.76126389032]
area = 100
  by: Ripley's K and L functions of a point pattern  (K(d) = A * (number of ordered pairs closer
than d) / n^2 with A the bounding-rectangle area (as pointpats))
  assumes: window = bounding rectangle of the points; pairs closer than d (strictly) are counted;
no edge correction
```

What the worked example means: The analysis computes Ripley's K statistic, which quantifies spatial clustering by counting pairs of points closer than the specified distance [1.5, 3, 12]. The result is scaled by the bounding rectangle area 100 and the number of points to produce a K value of [22.2222, 33.3333, 44.4444]. To stabilize variance for easier interpretation, the L function is derived as the square root of K divided by pi, resulting in [2.65962, 3.25735, 3.76126]. A higher L value relative to the expected baseline indicates that points are more clustered than would be expected under complete spatial randomness.

Standard deviational ellipse of a point pattern — `geo_standard_deviational_ellipse`

The semi-axes of the standard deviational ellipse of a point pattern (CrimeStat correction sqrt(2) sqrt(n/(n-2))) and the orientation of its major axis.

$$s_{x,y} = \sqrt{\frac{2n}{n-2}\lambda_{1,2}}, \quad \lambda_{1,2} = \frac{s_{xx}+s_{yy}}{2} \pm \sqrt{\frac{(s_{xx}-s_{yy})^2}{4} + s_{xy}^2}$$

symbol		meaning	SI unit
`x`	in	x coordinates	—
`y`	in	y coordinates	—
`sx`	out	semi-major axis	—
`sy`	out	semi-minor axis	—
`orientation`	out	bearing of the major axis, degrees clockwise from north (y axis), in [0, 180)	—

Applies when: n >= 3, points neither collinear nor circularly symmetric.

```
$ geospatial-analytics solve geo_standard_deviational_ellipse "x=[66.22, 22.52, 31.01, 9.47, 30.78,
75.21, 79.26, 8.23, 98.73, 89.78, 65.19]" "y=[32.54, 22.39, 81.21, 31.02, 60.1, 58.93, 7.68, 39.93,
77.17, 42.53, 92.08]"
sx = 50.13029459
sy = 37.9522267
orientation = 70.14367724
  by: Standard deviational ellipse of a point pattern  (eigenvalues of the (1/n) covariance of the
coordinates give the variances along the principal axes; axes = sqrt(2 n/(n-2) lambda) (CrimeStat);
orientation from the major eigenvector)
  assumes: n >= 3, points neither collinear nor circularly symmetric
```

What the worked example means: This calculation determines the spatial spread and directional bias of the point pattern by fitting an ellipse to the data. The resulting semi-major axis, measured as 50.1303, indicates the maximum extent of dispersion along the primary direction of elongation. Conversely, the semi-minor axis, recorded as 37.9522, shows the spread perpendicular to this main trend. The orientation value of 70.1437

degrees specifies the bearing of the major axis relative to north, revealing the dominant direction in which the points are aligned or dispersed.

Weighted mean centre and weighted standard distance — `geo_weighted_mean_center`

The weighted mean centre of a set of points (e.g. population-weighted centroid) and the weighted standard distance about it.

$$\bar{x}_w = \frac{\sum w_i x_i}{\sum w_i}, \quad SD_w = \sqrt{\frac{\sum w_i \left[(x_i - \bar{x}_w)^2 + (y_i - \bar{y}_w)^2\right]}{\sum w_i}}$$

symbol		meaning	SI unit
`x`	in	x coordinates	—
`y`	in	y coordinates	—
`w`	in	non-negative weights	—
`wmean_x`	out	x of the weighted mean centre	—
`wmean_y`	out	y of the weighted mean centre	—
`weighted_std_distance`	out	weighted standard distance	—

Applies when: non-negative weights, not all zero.

```
$ geospatial-analytics solve geo_weighted_mean_center "x=[0, 10, 20]" "y=[0, 0, 10]" "w=[100, 50,
50]"
wmean_x = 7.5
wmean_y = 2.5
weighted_std_distance = 9.354143467
  by: Weighted mean centre and weighted standard distance  (weighted means of the coordinates;
sqrt(sum w d^2 / sum w))
  assumes: non-negative weights, not all zero
```

What the worked example means: This calculation determines the weighted mean centre using the x coordinates [0, 10, 20] and y coordinates [0, 0, 10], adjusted by non-negative weights [100, 50, 50]. The resulting central point, 7.5 and 2.5, represents the balance point where heavier weights pull the location closer to them. The weighted standard distance, 9.35414, quantifies the average spread of the points from this centre, providing a measure of dispersion that accounts for the significance of each location.

Remote sensing (6)

At-sensor brightness temperature from thermal radiance — `geo_brightness_temperature`

The at-sensor brightness temperature of a thermal-infrared band from its spectral radiance and the band's calibration constants K1 and K2 (as in Landsat metadata).

$$T_B = \frac{K_2}{\ln\left(\frac{K_1}{L_\lambda} + 1\right)}$$

symbol		meaning	SI unit
`radiance`	in	spectral radiance (W/(m^2 sr um))	—
`K1`	in	band constant K1 (W/(m^2 sr um))	—
`K2`	in	band constant K2 (kelvin)	—
`T`	out	brightness temperature	K

Applies when: positive radiance and constants.

```
$ geospatial-analytics solve geo_brightness_temperature radiance=10.5 K1=774.8853 K2=1321.0789
T = 306.1739025 K
  by: At-sensor brightness temperature from thermal radiance  (inverted Planck relation T = K2 /
ln(K1/L + 1))
  radiance = 10.5   []
  K1 = 774.8853   []
  K2 = 1321.0789   []
  assumes: positive radiance and constants
```

What the worked example means: This calculation determines the physical heat equivalent of the light detected by the thermal sensor. By applying the inverted Planck relation to the measured spectral radiance of 10.5, combined with the specific band constants, the model outputs a brightness temperature of 306.174 K. This result translates raw intensity data into a tangible temperature value in Kelvin, enabling direct comparison with meteorological records or surface energy balance models.

Ground sample distance and swath of a nadir-looking camera — `geo_ground_sample_distance`

The ground sample distance (pixel footprint) and swath width of a nadir-looking frame camera or scanner from its pixel pitch, focal length, flying height above ground and number of pixels across track.

$$GSD = \frac{p\,H}{f}, \quad W = GSD \times N$$

symbol		meaning	SI unit
`pixel_pitch`	in	detector pixel pitch	m
`focal_length`	in	focal length	m
`altitude`	in	height above the ground	m
`pixels`	in	number of pixels across track	—
`gsd`	out	ground sample distance	m
`swath`	out	swath width	m

Applies when: flat ground, nadir view.

```
$ geospatial-analytics solve geo_ground_sample_distance "pixel_pitch=0.0024 mm" "focal_length=8.8
mm" "altitude=120 m" pixels=5472
gsd = 0.03272727273 m
swath = 179.0836364 m
  by: Ground sample distance and swath of a nadir-looking camera  (similar triangles: GSD = pixel
pitch x height / focal length; swath = GSD x pixels)
  pixel_pitch = 2.4e-06 m   []
  focal_length = 0.0088 m   []
  altitude = 120 m   []
  assumes: flat ground, nadir view
```

What the worked example means: This calculation determines the spatial resolution and coverage area of a nadir-looking camera. Using similar triangles, it computes the ground sample distance from the pixel pitch, flying height, and focal length. The total swath width is then derived by multiplying this ground sample distance by the number of pixels across track. For the given inputs, the resulting ground sample distance is 0.0327273 m and the swath width is 179.084 m.

Thematic map accuracy: overall accuracy, kappa, producer's and user's accuracy — `geo_classification_accuracy`

The accuracy assessment of a classified map from its error (confusion) matrix: overall accuracy, Cohen's kappa, and the producer's and user's accuracy of each class.

$$OA = \frac{\sum_i n_{ii}}{N}, \quad \kappa = \frac{p_o - p_e}{1 - p_e}, \quad PA_j = \frac{n_{jj}}{n_{+j}}, \quad UA_i = \frac{n_{ii}}{n_{i+}}$$

symbol		meaning	SI unit
`matrix`	in	confusion matrix of sample counts: rows = map (classified) classes, columns = reference classes	—
`overall`	out	overall accuracy	—
`kappa`	out	Cohen's kappa	—
`producers`	out	producer's accuracy of each class (1 - omission error)	—
`users`	out	user's accuracy of each class (1 - commission error)	—

Applies when: square matrix of whole counts; every class sampled in the map and the reference.

```
$ geospatial-analytics solve geo_classification_accuracy "matrix=[[65, 4, 22, 24], [6, 81, 5, 8],
[0, 11, 85, 19], [4, 7, 3, 90]]"
overall = 0.7396313364
kappa = 0.6535162708
producers = [0.866666666667,0.78640776699,0.739130434783,0.63829787234]
users = [0.565217391304,0.81,0.739130434783,0.865384615385]
  by: Thematic map accuracy: overall accuracy, kappa, producer's and user's accuracy  (overall =
trace/N; kappa = (po - pe)/(1 - pe) with pe from the row and column totals; producer's =
diagonal/column total, user's = diagonal/row total)
  assumes: square matrix of whole counts; every class sampled in the map and the reference
```

What the worked example means: This calculation derives the accuracy metrics for a classified map from its confusion matrix. It computes the overall accuracy as the proportion of correctly classified samples, represented here by 0.739631. The Cohen's kappa coefficient, shown as 0.653516, measures agreement

beyond what would be expected by chance. Additionally, it provides class-specific producer's accuracies, listed as [0.866667, 0.786408, 0.73913, 0.638298], which indicate how often reference points were correctly mapped, and user's accuracies, listed as [0.565217, 0.81, 0.73913, 0.865385], which show how often map pixels correspond to the correct reference class.

Top-of-atmosphere reflectance from at-sensor radiance — `geo_toa_reflectance`

The top-of-atmosphere (planetary) reflectance of a pixel from its at-sensor spectral radiance, the band's mean exoatmospheric solar irradiance ESUN, the day of year and the sun elevation, with the Earth-Sun distance.

$$\rho_{TOA} = \frac{\pi L_\lambda d^2}{ESUN_\lambda \cos\theta_s}, \quad \frac{1}{d^2} = 1.00011 + 0.034221\cos\Gamma + 0.00128\sin\Gamma + 0.000719\cos 2\Gamma + 0.000($$

symbol		meaning	SI unit
`radiance`	in	at-sensor spectral radiance (W/(m^2 sr um))	—
`esun`	in	band mean exoatmospheric solar irradiance ESUN (W/(m^2 um))	—
`doy`	in	day of year (1-366)	—
`sun_elevation`	in	sun elevation angle (decimal degrees)	—
`earth_sun_distance`	out	Earth-Sun distance (astronomical units)	—
`reflectance`	out	top-of-atmosphere reflectance	—

Applies when: sun above the horizon.

```
$ geospatial-analytics solve geo_toa_reflectance radiance=85 esun=1536 doy=172 sun_elevation=60
earth_sun_distance = 1.016687196
reflectance = 0.2075017002
  by: Top-of-atmosphere reflectance from at-sensor radiance  (rho = pi L d^2 / (ESUN sin(sun
elevation)); d^2 from Spencer's (1971) Fourier series for (r0/r)^2)
  radiance = 85   []
  esun = 1536   []
  sun_elevation = 60   []
  assumes: sun above the horizon
```

What the worked example means: This calculation determines the top-of-atmosphere reflectance for a pixel using its measured at-sensor spectral radiance, the band-specific exoatmospheric solar irradiance, and the sun elevation angle. It incorporates the Earth-Sun distance correction based on the day of year to account for seasonal variations in solar intensity. The resulting value of 0.207502 represents the normalized brightness of the surface as seen from space, allowing for comparison across different times and locations by removing atmospheric and geometric effects.

Vegetation indices: NDVI, EVI, SAVI, OSAVI, MSAVI and EVI2 from band reflectances — `geo_vegetation_indices`

The common spectral vegetation indices of a pixel from its red, near-infrared and blue surface reflectances, with a stated soil adjustment factor L for SAVI (EVI uses its standard coefficients G = 2.5, C1 = 6, C2 = 7.5, L = 1).

$$NDVI = \frac{\rho_N - \rho_R}{\rho_N + \rho_R},\ EVI = 2.5\frac{\rho_N - \rho_R}{\rho_N + 6\rho_R - 7.5\rho_B + 1},\ SAVI = (1+L)\frac{\rho_N - \rho_R}{\rho_N + \rho_R + L}$$

symbol		meaning	SI unit
`red`	in	red reflectance (0-1)	—
`nir`	in	near-infrared reflectance (0-1)	—
`blue`	in	blue reflectance (0-1)	—
`L`	in	SAVI soil adjustment factor (0.5 is usual)	—
`NDVI`	out	(NIR - R)/(NIR + R)	—
`EVI`	out	2.5 (NIR - R)/(NIR + 6R - 7.5B + 1)	—
`SAVI`	out	(1 + L)(NIR - R)/(NIR + R + L)	—
`OSAVI`	out	(NIR - R)/(NIR + R + 0.16)	—
`MSAVI`	out	(2NIR + 1 - sqrt((2NIR + 1)^2 - 8(NIR - R)))/2	—
`EVI2`	out	2.5 (NIR - R)/(NIR + 2.4R + 1)	—

Applies when: reflectances between 0 and 1; positive denominators.

```
$ geospatial-analytics solve geo_vegetation_indices red=0.08 nir=0.45 blue=0.04 L=0.5
NDVI = 0.6981132075
EVI = 0.5674846626
SAVI = 0.5388349515
OSAVI = 0.5362318841
MSAVI = 0.5468871126
EVI2 = 0.5633373934
  by: Vegetation indices: NDVI, EVI, SAVI, OSAVI, MSAVI and EVI2 from band reflectances  (the
published formulas (Rouse 1974; Huete 1988, 2002; Rondeaux 1996; Qi 1994; Jiang 2008))
  red = 0.08   []
  nir = 0.45   []
  blue = 0.04   []
  L = 0.5   []
  assumes: reflectances between 0 and 1; positive denominators
```

What the worked example means: This calculation determines six common spectral vegetation indices using the red, near-infrared, and blue surface reflectances of a single pixel. The Normalized Difference Vegetation Index, or NDVI, is computed as 0.698113 by comparing the difference between near-infrared and red reflectance to their sum. The Enhanced Vegetation Index, or EVI, yields 0.567485 by incorporating blue reflectance to correct for atmospheric effects and soil brightness. The Soil Adjusted Vegetation Index, or SAVI, results in 0.538835 by applying a soil adjustment factor L of 0.5 to minimize background interference. Optimized SAVI gives 0.536232 with a fixed factor, while Modified SAVI produces 0.546887 without needing an explicit soil factor. Finally, the Two-Band Enhanced Vegetation Index, or EVI2, calculates 0.563337 using only red and near-infrared bands. These values indicate the density and health of vegetation at that location.

Water, moisture, burn and built-up indices: NDWI, MNDWI, NDMI, NBR and NDBI — `geo_water_burn_built_indices`

Normalized-difference indices for open water (NDWI, MNDWI), vegetation moisture (NDMI), burn severity (NBR) and built-up land (NDBI) from green, near-infrared and two shortwave-infrared reflectances.

$$ND(a,b) = \frac{\rho_a - \rho_b}{\rho_a + \rho_b}$$

symbol		meaning	SI unit
`green`	in	green reflectance (0-1)	—
`nir`	in	near-infrared reflectance (0-1)	—
`swir1`	in	shortwave infrared 1 (about 1.6 um) reflectance	—
`swir2`	in	shortwave infrared 2 (about 2.2 um) reflectance	—
`NDWI`	out	(G - NIR)/(G + NIR)	—
`MNDWI`	out	(G - SWIR1)/(G + SWIR1)	—
`NDMI`	out	(NIR - SWIR1)/(NIR + SWIR1)	—
`NBR`	out	(NIR - SWIR2)/(NIR + SWIR2)	—
`NDBI`	out	(SWIR1 - NIR)/(SWIR1 + NIR)	—

Applies when: reflectances between 0 and 1, band sums non-zero.

```
$ geospatial-analytics solve geo_water_burn_built_indices green=0.08 nir=0.03 swir1=0.02 swir2=0.01
NDWI = 0.4545454545
MNDWI = 0.6
NDMI = 0.2
NBR = 0.5
NDBI = -0.2
  by: Water, moisture, burn and built-up indices: NDWI, MNDWI, NDMI, NBR and NDBI  (normalized
differences of the band pairs (McFeeters 1996; Xu 2006; Gao 1996; Key and Benson 2006; Zha 2003))
  green = 0.08   []
  nir = 0.03   []
  swir1 = 0.02   []
  swir2 = 0.01   []
  assumes: reflectances between 0 and 1, band sums non-zero
```

What the worked example means: This analysis calculates five spectral indices using the normalized difference formula. The inputs are green reflectance of 0.08, near-infrared reflectance of 0.03, shortwave infrared one reflectance of 0.02, and shortwave infrared two reflectance of 0.01. These values produce an NDWI of 0.454545 for open water detection, an MNDWI of 0.6 which reduces built-up noise in water mapping, an NDMI of 0.2 indicating vegetation moisture content, an NBR of 0.5 assessing burn severity, and an NDBI of -0.2 highlighting urban surfaces.

Spatial statistics (10)

Bivariate Moran's I (spatial cross-correlation) — `geo_bivariate_morans_i`

The bivariate Moran's I between a variable x and the spatial lag of a second variable y with row-standardized weights (how x at a location relates to y around it).

$$I_{xy} = \frac{\sum_i z_{x,i} \sum_j w_{ij} z_{y,j}}{n - 1}$$

symbol		meaning	SI unit
`x`	in	first variable at the n locations	—
`y`	in	second variable at the n locations	—
`W`	in	spatial weights matrix (row-standardized internally)	—
`I_xy`	out	bivariate Moran's I	—

Applies when: n >= 4, neither variable constant; no observation without neighbours.

```
$ geospatial-analytics solve geo_bivariate_morans_i "x=[8, 6, 6, 3, 2, 1]" "y=[7, 7, 5, 4, 1, 2]"
"W=[[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0,
1], [0, 0, 0, 0, 1, 0]]"
I_xy = 0.8917603446
  by: Bivariate Moran's I (spatial cross-correlation)  (I_xy = zx' W zy / (n - 1) with both
variables standardized (sample standard deviation) and row-standardized W)
  assumes: n >= 4, neither variable constant; no observation without neighbours
```

What the worked example means: This calculation determines the bivariate Moran's I, which measures how the first variable at each location relates to the average value of the second variable in its neighboring areas. The result, 0.89176, reflects this spatial association after standardizing both input variables and using row-standardized weights derived from the provided spatial configuration.

Distance-band spatial weights — `geo_distance_band_weights`

The binary distance-band weights matrix of a set of points (neighbours within a threshold distance), the number of neighbours of each point, the total weight and the number of islands.

$$w_{ij} = \begin{cases} 1 & d_{ij} \le \delta,\ i \ne j \\ 0 & \text{otherwise} \end{cases}$$

symbol		meaning	SI unit
`x`	in	x coordinates	—
`y`	in	y coordinates	—
`threshold`	in	distance threshold (coordinate units)	—
`W`	out	weights matrix (n x n, 0/1)	—
`neighbours`	out	number of neighbours of each point	—
`s0`	out	sum of all weights	—
`islands`	out	number of points without neighbours	—

Applies when: threshold > 0.

```
$ geospatial-analytics solve geo_distance_band_weights "x=[0, 1, 3, 6, 10]" "y=[0, 0, 0, 0, 0]"
threshold=2.5
W = [[0.0,1.0,0.0,0.0,0.0],[1.0,0.0,1.0,0.0,0.0],[0.0,1.0,0.0,0.0,0.0],[0.0,0.0,0.0,0.0,0.0],
[0.0,0.0,0.0,0.0,0.0]]
neighbours = [1.0,2.0,1.0,0.0,0.0]
s0 = 4
islands = 2
  by: Distance-band spatial weights  (w_ij = 1 when 0 < d_ij <= threshold)
  threshold = 2.5   []
  assumes: threshold > 0
```

What the worked example means: This calculation constructs a binary spatial weights matrix where each point is connected to every other point within a distance of 2.5. The resulting matrix W contains ones for neighbors and zeros otherwise. Diagnostics show that the total number of connections, represented by s0, is 4, while 2 points have no neighbors at all. The number of neighbors for each specific location is detailed in the output list neighbours.

Geary's C spatial autocorrelation with normal and randomization inference — `geo_gearys_c`

Geary's contiguity ratio C of a variable with a given spatial weights matrix, its variance under normality and randomization, and the z-scores (C < 1 positive, C > 1 negative autocorrelation).

$$C = \frac{(n-1)\sum_i \sum_j w_{ij}(y_i - y_j)^2}{2S_0 \sum_i (y_i - \bar{y})^2}, \quad V_N[C] = \frac{(2S_1 + S_2)(n-1) - 4S_0^2}{2(n+1)S_0^2}$$

symbol		meaning	SI unit
`y`	in	values at the n locations	—
`W`	in	spatial weights matrix (n x n, zero diagonal, row i lists the weights of i's neighbours)	—
`transform`	in	row-standardize the weights (row) or use them as given (none)	—
`C`	out	Geary's C	—
`VC_norm`	out	variance under normality	—
`z_norm`	out	z-score under normality	—
`VC_rand`	out	variance under randomization	—
`z_rand`	out	z-score under randomization	—

Applies when: n >= 4, values not all equal; no observation without neighbours.

```
$ geospatial-analytics solve geo_gearys_c "y=[8, 6, 6, 3, 2, 1]" "W=[[0, 1, 0, 0, 0, 0], [1, 0, 1,
0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]]"
transform=row
C = 0.1953125
VC_norm = 0.1111111111
z_norm = -2.4140625
VC_rand = 0.1322312128
z_rand = -2.212892233
  by: Geary's C spatial autocorrelation with normal and randomization inference  (C = (n-1) sum
w_ij (y_i - y_j)^2 / (2 S0 sum z^2); Cliff-Ord moments (E[C] = 1))
  transform = row
  assumes: n >= 4, values not all equal; no observation without neighbours
```

What the worked example means: Geary's C measures spatial autocorrelation by comparing differences between neighboring locations to the overall variance. The computed value of 0.195312 reflects how much adjacent areas differ from each other relative to the total variation in the data. A result less than one implies that neighboring regions have similar values, while a result greater than one indicates dissimilarity among neighbors.

Getis-Ord general G (global high/low clustering) — `geo_getis_ord_general_g`

The Getis-Ord general G statistic of a positive variable with binary spatial weights, its expectation and variance under randomization and the z-score (G above its expectation: high values cluster).

$$G = \frac{\sum_i \sum_{j \neq i} w_{ij} y_i y_j}{\sum_i \sum_{j \neq i} y_i y_j}, \quad E[G] = \frac{S_0}{n(n-1)}$$

symbol		meaning	SI unit
`y`	in	positive values at the n locations	—
`W`	in	binary spatial weights matrix (0/1, zero diagonal)	—
`G`	out	general G	—
`EG`	out	expected G = S0/(n(n-1))	—
`VG`	out	variance of G under randomization	—
`z_norm`	out	z-score	—

Applies when: positive values, n >= 4; binary weights, no observation without neighbours.

```
$ geospatial-analytics solve geo_getis_ord_general_g "y=[10, 12, 11, 2, 3, 2]" "W=[[0, 1, 1, 0, 0,
0], [1, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 1], [0, 0, 0, 1, 0, 1], [0, 0, 0, 1, 1,
0]]"
G = 0.6568144499
EG = 0.4666666667
VG = 0.00892117238
z_norm = 2.01316931
  by: Getis-Ord general G (global high/low clustering)  (G = sum_{i != j} w_ij y_i y_j / sum_{i !=
j} y_i y_j; Getis-Ord (1992) moments)
  assumes: positive values, n >= 4; binary weights, no observation without neighbours
```

What the worked example means: This analysis calculates the Getis-Ord general G statistic, which measures global spatial autocorrelation for positive variables. Using input values [10, 12, 11, 2, 3, 2] and binary spatial weights [0 1 1 0 0 0; 1 0 1 0 0 0; 1 1 0 1 0 0; 0 0 1 0 1 1; 0 0 0 1 0 1; 0 0 0 1 1 0], the model computes the observed statistic 0.656814. This is then compared against the theoretical expectation 0.466667 derived from randomization moments. A z-score of 2.01317 indicates whether high or low values are clustered more than would be expected by chance, with a variance of 0.00892117 quantifying the dispersion of this measure.

Global Moran's I spatial autocorrelation with normal and randomization inference — `geo_morans_i`

Global Moran's I of a variable observed at n locations with a given spatial weights matrix, its expectation and variance under normality and under randomization, z-scores and two-sided p-values.

$$I = \frac{n}{S_0}\frac{\sum_i\sum_j w_{ij}z_iz_j}{\sum_i z_i^2},\quad E[I] = -\frac{1}{n-1},\quad V_N[I] = \frac{n^2S_1 - nS_2 + 3S_0^2}{(n^2-1)S_0^2} - E[I]^2$$

symbol		meaning	SI unit
`y`	in	values at the n locations	—
`W`	in	spatial weights matrix (n x n, zero diagonal, row i lists the weights of i's neighbours)	—
`transform`	in	row-standardize the weights (row) or use them as given (none)	—
`I`	out	Moran's I	—
`EI`	out	expected I = -1/(n-1)	—
`VI_norm`	out	variance under normality	—
`z_norm`	out	z-score under normality	—
`p_norm`	out	two-sided p-value under normality	—
`VI_rand`	out	variance under randomization	—
`z_rand`	out	z-score under randomization	—
`p_rand`	out	two-sided p-value under randomization	—

Applies when: n >= 4, values not all equal; no observation without neighbours.

```
$ geospatial-analytics solve geo_morans_i "y=[8, 6, 6, 3, 2, 1]" "W=[[0, 1, 0, 0, 0, 0], [1, 0, 1,
0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]]"
transform=row
I = 0.65625
EI = -0.2
VI_norm = 0.140952381
z_norm = 2.280680117
p_norm = 0.02256738258
VI_rand = 0.1817926233
z_rand = 2.008225402
p_rand = 0.04461934483
  by: Global Moran's I spatial autocorrelation with normal and randomization inference  (I = (n/S0)
sum w_ij z_i z_j / sum z_i^2 with z = y - mean; Cliff-Ord moments with S0, S1, S2 and the sample
kurtosis)
  transform = row
  assumes: n >= 4, values not all equal; no observation without neighbours
```

What the worked example means: This calculation determines the global Moran's I statistic, which measures spatial autocorrelation across all locations. It uses the observed values and the spatial weights matrix to compute a value of 0.65625. The expected value under randomness is -0.2, while the variance under normality is 0.140952. A z-score of 2.28068 and a p-value of 0.0225674 help assess whether the observed clustering is statistically significant compared to a random distribution.

Join count statistics of a binary map (BB, BW, WW) — `geo_join_counts`

The numbers of black-black, black-white and white-white joins between neighbouring areas of a binary (presence/absence) map, and the total number of joins.

$$BB = \tfrac{1}{2}\sum_{i,j} w_{ij} y_i y_j,\; BW = \tfrac{1}{2}\sum_{i,j} w_{ij}(y_i - y_j)^2,\; WW = \tfrac{1}{2}\sum_{i,j} w_{ij}(1-y_i)(1-y_j)$$

symbol		meaning	SI unit
`y`	in	binary variable at the n areas (1 black, 0 white)	—
`W`	in	binary symmetric contiguity matrix (0/1, zero diagonal)	—
`BB`	out	black-black joins	—
`BW`	out	black-white joins	—
`WW`	out	white-white joins	—
`J`	out	total joins	—

Applies when: binary symmetric weights.

```
$ geospatial-analytics solve geo_join_counts "y=[1, 1, 0, 0, 1, 0]" "W=[[0, 1, 1, 0, 0, 0], [1, 0,
1, 1, 0, 0], [1, 1, 0, 0, 1, 0], [0, 1, 0, 0, 1, 1], [0, 0, 1, 1, 0, 1], [0, 0, 0, 1, 1, 0]]"
BB = 1
BW = 6
WW = 1
J = 8
  by: Join count statistics of a binary map (BB, BW, WW)  (count each neighbouring pair once by the
colours of its two areas)
  assumes: binary symmetric weights
```

What the worked example means: This calculation determines the spatial configuration of a binary map by counting adjacent area pairs based on their colors. It computes the number of connections between two black areas as 1, between a black and a white area as 6, and between two white areas as 1. The sum of these categories yields the total number of neighboring pairs, reported here as 8. A high count of like-color joins relative to unlike-color joins indicates spatial clustering, whereas a predominance of unlike-color joins suggests a dispersed or mixed pattern across the study region.

k-nearest-neighbour spatial weights — `geo_knn_weights`

The binary k-nearest-neighbour weights matrix of a set of points (row i marks the k nearest points to point i), its total weight and whether it is symmetric.

$$w_{ij} = 1 \iff j \in N_k(i)$$

symbol		meaning	SI unit
`x`	in	x coordinates	—
`y`	in	y coordinates	—
`k`	in	number of neighbours	—
`W`	out	weights matrix (n x n, 0/1)	—
`s0`	out	sum of all weights (n k)	—
`symmetric`	out	whether W is symmetric	—

Applies when: distinct points; 1 <= k < n, no tie at the k-th neighbour.

```
$ geospatial-analytics solve geo_knn_weights "x=[0, 1, 3.5, 6, 10]" "y=[0, 0, 0, 0, 0.5]" k=2
W = [[0.0,1.0,1.0,0.0,0.0],[1.0,0.0,1.0,0.0,0.0],[0.0,1.0,0.0,1.0,0.0],[0.0,0.0,1.0,0.0,1.0],
[0.0,0.0,1.0,1.0,0.0]]
s0 = 10
symmetric = false
  by: k-nearest-neighbour spatial weights  (sort each point's distances to the others and mark the
k smallest)
  assumes: distinct points; 1 <= k < n, no tie at the k-th neighbour
```

What the worked example means: This analysis computes the binary k-nearest neighbor spatial weights matrix [[0, 1, 1, 0, 0], [1, 0, 1, 0, 0], [0, 1, 0, 1, 0], [0, 0, 1, 0, 1], [0, 0, 1, 1, 0]] for the given coordinates. Each row indicates which points are among the nearest neighbors to that location, based on distance sorting. The total sum of all connections is reported as 10, while the symmetry status no reveals whether the relationship between every pair of points is mutual.

Local Getis-Ord Gi and Gi* hot-spot statistics — `geo_local_getis_ord_gi`

The local Getis-Ord statistic of every location (Gi* includes the location itself, Gi excludes it) with binary weights, and its z-score for hot-spot (positive) and cold-spot (negative) mapping.

$$G_i^* = \frac{\sum_j w_{ij} y_j}{\sum_j y_j}, \quad Z_i = \frac{G_i - W_i/N}{\sqrt{\frac{W_i(N-W_i)}{(N-1)N^2}\,\frac{s^2}{\bar{y}^2}}}$$

symbol		meaning	SI unit
`y`	in	positive values at the n locations	—
`W`	in	binary spatial weights matrix (0/1, zero diagonal)	—
`statistic`	in	Gi* (the location counts as its own neighbour) or Gi	—
`Gs`	out	local G of each location	—
`Zs`	out	z-score of each location	—

Applies when: positive values, n >= 4; binary weights, no observation without neighbours.

```
$ geospatial-analytics solve geo_local_getis_ord_gi "y=[10, 12, 11, 2, 3, 2]" "W=[[0, 1, 1, 0, 0, 0], [1, 0, 1, 0, 0, 0], [1, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 1], [0, 0, 0, 1, 0, 1], [0, 0, 0, 1, 1, 0]]" statistic=gi_star
Gs = [0.825,0.825,0.875,0.45,0.175,0.175]
Zs = [2.21006630298,2.21006630298,1.50264699011,-1.56275286971,-2.21006630298,-2.21006630298]
  by: Local Getis-Ord Gi and Gi* hot-spot statistics  (G_i = sum_j w_ij y_j / sum_j y_j (j != i for Gi); E = W_i/N, V = W_i (N - W_i)/(N-1)/N^2 * s^2/ybar^2 with the values other than i for Gi)
  statistic = gi_star
  assumes: positive values, n >= 4; binary weights, no observation without neighbours
```

What the worked example means: This calculation determines the Getis-Ord statistic for a specific site, yielding a raw value of [0.825, 0.825, 0.875, 0.45, 0.175, 0.175]. Comparing this against the expected distribution derived from the input values [10, 12, 11, 2, 3, 2] and the connectivity defined by weights [0 1 1 0 0 0; 1 0 1 0 0 0; 1 1 0 1 0 0; 0 0 1 0 1 1; 0 0 0 1 0 1; 0 0 0 1 1 0] gives a z-score of [2.21007, 2.21007, 1.50265, -1.56275, -2.21007, -2.21007]. A positive z-score suggests a hot spot where high values cluster together, while a negative score indicates a cold spot with low values concentrated in the neighborhood. The result depends on whether the location itself is included in the summation, as specified by the chosen statistic type.

Local Moran's I (LISA) and Moran scatterplot quadrants — `geo_local_morans_i`

The local Moran statistic of every location with row-standardized weights, and its Moran scatterplot quadrant: 1 high-high, 2 low-high, 3 low-low, 4 high-low.

$$I_i = (n-1)\frac{z_i \sum_j w_{ij} z_j}{\sum_k z_k^2}$$

symbol		meaning	SI unit
`y`	in	values at the n locations	—
`W`	in	spatial weights matrix (row-standardized internally)	—
`Is`	out	local Moran's I of each location	—
`quadrant`	out	quadrant of each location (1 HH, 2 LH, 3 LL, 4 HL)	—

Applies when: n >= 4, values not all equal; no observation without neighbours.

```
$ geospatial-analytics solve geo_local_morans_i "y=[8, 6, 6, 3, 2, 1]" "W=[[0, 1, 0, 0, 0, 0], [1, 0, 1, 0, 0, 0], [0, 1, 0, 1, 0, 0], [0, 0, 1, 0, 1, 0], [0, 0, 0, 1, 0, 1], [0, 0, 0, 0, 1, 0]]"
Is = [0.818452380952,0.595238095238,0.0372023809524,0.0595238095238,0.729166666667,1.04166666667]
quadrant = [1.0,1.0,1.0,3.0,3.0,3.0]
  by: Local Moran's I (LISA) and Moran scatterplot quadrants  (I_i = (n - 1) z_i sum_j w_ij z_j / sum_k z_k^2 with z = y - mean and row-standardized w (as esda))
  assumes: n >= 4, values not all equal; no observation without neighbours
```

What the worked example means: This calculation determines the local spatial autocorrelation for each location using the input values [8, 6, 6, 3, 2, 1] and the spatial weights matrix [0 1 0 0 0 0; 1 0 1 0 0 0; 0 1 0 1 0 0; 0 0 1 0 1 0; 0 0 0 1 0 1; 0 0 0 0 1 0]. The resulting local Moran's I statistics [0.818452, 0.595238, 0.0372024, 0.0595238, 0.729167, 1.04167] indicate how strongly each point correlates with its neighbors, while the assigned quadrants [1, 1, 1, 3, 3, 3] classify these points into high-high, low-high, low-low, or high-low categories based on their value relative to the mean.

Spatial lag of a variable — `geo_spatial_lag`

The spatial lag W y of a variable: for row-standardized weights, the average of each location's neighbours; otherwise the weighted sum.

$$[Wy]_i = \sum_j w_{ij} y_j$$

symbol		meaning	SI unit
`y`	in	values at the n locations	—
`W`	in	spatial weights matrix (n x n, zero diagonal, row i lists the weights of i's neighbours)	—
`transform`	in	row-standardize the weights (row) or use them as given (none)	—
`lag`	out	spatial lag at each location	—

Applies when: no observation without neighbours.

```
$ geospatial-analytics solve geo_spatial_lag "y=[8, 6, 6, 3]" "W=[[0, 1, 0, 1], [1, 0, 1, 0], [0,
1, 0, 1], [1, 0, 1, 0]]" transform=row
lag = [4.5,7.0,4.5,7.0]
  by: Spatial lag of a variable  (lag_i = sum_j w_ij y_j (weights row-standardized first if asked))
  transform = row
  assumes: no observation without neighbours
```

What the worked example means: This result represents the spatial lag, which aggregates information from neighboring locations. For location i, it computes [4.5, 7, 4.5, 7] by applying the weights matrix W to the variable y. If row standardization was requested, this value reflects the average of neighbors; otherwise, it is a weighted sum. The calculation relies on the input values [8, 6, 6, 3] and the structure defined in [0 1 0 1; 1 0 1 0; 0 1 0 1; 1 0 1 0].

Terrain analysis (8)

D8 flow accumulation of a small DEM — `geo_d8_flow_accumulation`

The number of cells draining through each cell (the cell itself included) under D8 flow directions, and the largest accumulation (the main outlet).

$$A(c) = 1 + \sum_{u \to c} A(u)$$

symbol		meaning	SI unit
`dem`	in	elevations (row 0 to the north), at most 900 cells	—
`cell_size`	in	cell size	m
`accumulation`	out	contributing cells of each cell, itself included (matrix)	—
`max_accumulation`	out	largest accumulation	—

Applies when: D8 directions as in geo_d8_flow_direction.

```
$ geospatial-analytics solve geo_d8_flow_accumulation "dem=[[78, 72, 69, 71], [74, 67, 56, 49],
[69, 53, 44, 37], [64, 58, 55, 22]]" "cell_size=10 m"
accumulation = [[1.0,1.0,1.0,1.0],[1.0,2.0,2.0,3.0],[1.0,4.0,8.0,6.0],[1.0,1.0,1.0,16.0]]
max_accumulation = 16
  by: D8 flow accumulation of a small DEM  (D8 directions, then accumulation in topological order
of the flow graph (Kahn's algorithm))
  cell_size = 10 m   []
  assumes: D8 directions as in geo_d8_flow_direction
```

What the worked example means: This calculation determines how many cells drain through each location on the digital elevation model, including the cell itself. The result shows that the largest accumulation value is 16, which identifies the primary stream network outlet. This metric helps in understanding hydrological connectivity and flood risk by quantifying the contributing area upstream of each grid cell.

D8 flow direction of every cell of a small DEM — `geo_d8_flow_direction`

The D8 (steepest-descent) flow direction of every cell of a digital elevation model, as ESRI codes (1 E, 2 SE, 4 S, 8 SW, 16 W, 32 NW, 64 N, 128 NE; 0 where no neighbour inside the raster is lower).

$$d^{*} = \arg\max_{k} \frac{z_0 - z_k}{\ell_k}, \quad \ell_k \in \{\Delta, \sqrt{2}\Delta\}$$

symbol		meaning	SI unit
`dem`	in	elevations (row 0 to the north), at most 900 cells	—
`cell_size`	in	cell size	m
`direction`	out	D8 code of each cell (matrix)	—

Applies when: square cells; diagonal distance sqrt(2) cells; ties go to the first direction in the order E, SE, S, SW, W, NW, N, NE.

```
$ geospatial-analytics solve geo_d8_flow_direction "dem=[[78, 72, 69, 71], [74, 67, 56, 49], [69,
53, 44, 37], [64, 58, 55, 22]]" "cell_size=10 m"
direction = [[2.0,2.0,2.0,4.0],[2.0,2.0,2.0,4.0],[1.0,1.0,2.0,4.0],[128.0,128.0,1.0,0.0]]
  by: D8 flow direction of every cell of a small DEM  (for each cell the neighbour with the largest
drop per unit distance (positive drops only))
  cell_size = 10 m   []
  assumes: square cells; diagonal distance sqrt(2) cells; ties go to the first direction in the
order E, SE, S, SW, W, NW, N, NE
```

What the worked example means: This calculation determines the steepest descent path for water flow across every cell of the digital elevation model. By comparing the elevation drop to each of the eight neighboring cells, adjusted for distance, the algorithm assigns a specific code to indicate where water will move next. The resulting matrix [[2, 2, 2, 4], [2, 2, 2, 4], [1, 1, 2, 4], [128, 128, 1, 0]] contains these ESRI-style codes for all input elevations [78 72 69 71; 74 67 56 49; 69 53 44 37; 64 58 55 22], using the grid spacing 10 m to normalize the gradient.

Hillshade (analytical relief shading) of a 3x3 elevation window — `geo_hillshade`

The illumination (cosine of the solar incidence angle on the terrain) and the 0-255 hillshade value at the centre of a 3 x 3 elevation window for a sun azimuth and altitude.

$$\cos i = \cos Z \cos S + \sin Z \sin S \cos(A - \alpha), \quad H = 255 \max(0, \cos i)$$

symbol		meaning	SI unit
`window`	in	3 x 3 elevations in metres, row 0 to the north	—
`cell_size`	in	cell size	m
`sun_azimuth`	in	sun azimuth, clockwise from north (decimal degrees)	—
`sun_altitude`	in	sun altitude above the horizon (decimal degrees)	—
`illumination`	out	cos(incidence angle) (negative in self-shadow)	—
`hillshade`	out	255 x max(0, illumination)	—

Applies when: row 0 is the northern row; no cast shadows.

```
$ geospatial-analytics solve geo_hillshade "window=[[2452.25, 2463, 2473.75], [2450, 2461, 2472],
[2447.75, 2459, 2470.25]]" "cell_size=5 m" sun_azimuth=315 sun_altitude=45
illumination = 0.656098596
hillshade = 167.305142
  by: Hillshade (analytical relief shading) of a 3x3 elevation window  (Horn slope and aspect, then
cos i = cos(Zenith) cos(Slope) + sin(Zenith) sin(Slope) cos(Azimuth - Aspect))
  cell_size = 5 m   []
  sun_azimuth = 315   []
  sun_altitude = 45   []
  assumes: row 0 is the northern row; no cast shadows
```

What the worked example means: This calculation determines the brightness of a terrain surface at the center of a three by three elevation window. It uses Horn's method to estimate slope and aspect, then combines these with the sun's position to find the cosine of the incidence angle. The resulting illumination value of 0.656099 represents how directly sunlight strikes the ground, which is scaled to produce the final hillshade intensity of 167.305. A higher value indicates a surface facing the light source more directly, while lower values suggest shadowed or steeply angled slopes.

Line-of-sight visibility along a terrain profile (with Earth curvature and refraction) — `geo_line_of_sight_profile`

Which points along a terrain profile are visible from an observer at its start, for given observer and target heights above the ground, optionally with the Earth-curvature and refraction correction.

$$\text{visible}(i) \iff z_j < z_{eye} + (z_i + h_t - z_{eye})\frac{d_j}{d_i} \; \forall\, 0 < j < i, \quad \Delta z_{curv} = \frac{d^2(1-k)}{2R}$$

symbol		meaning	SI unit
`d`	in	distances along the profile in metres, starting at 0 (the observer)	—
`z`	in	ground elevations in metres	—
`observer_height`	in	eye height above the ground	m
`target_height`	in	target height above the ground	m
`curvature`	in	none, or the drop d^2 (1 - k)/(2R) with R = 6371 km and k = 0.13	—
`visible`	out	1 if the target over each point is visible, else 0 (the observer's own point is 1)	—

symbol		meaning	SI unit
`n_visible`	out	number of visible points beyond the observer	—

Applies when: distances increase from 0; positive observer and target heights.

```
$ geospatial-analytics solve geo_line_of_sight_profile "d=[0, 100, 200, 300, 400, 500]" "z=[100,
104, 112, 103, 101, 109]" "observer_height=2 m" "target_height=2 m" curvature=none
visible = [1.0,1.0,1.0,0.0,0.0,0.0]
n_visible = 2
  by: Line-of-sight visibility along a terrain profile (with Earth curvature and refraction)  (a
target is visible when every intermediate ground point lies strictly below the sight line from the
eye to the target)
  observer_height = 2 m   []
  target_height = 2 m   []
  curvature = none
  assumes: distances increase from 0; positive observer and target heights
```

What the worked example means: This calculation assesses visibility along a terrain profile by comparing ground elevations against the straight-line path from the observer's eye to each target point. Using distances starting at [0, 100, 200, 300, 400, 500], ground heights [100, 104, 112, 103, 101, 109], an observer height of 2 m, and a target height of 2 m, the method checks if any intermediate terrain obstructs the view. The result indicates that 2 points are visible, incorporating a curvature correction of none to account for Earth shape and refraction.

Slope and aspect of a 3x3 elevation window (Horn's method) — `geo_slope_aspect_horn`

The slope (degrees and per cent) and aspect (downslope direction, degrees clockwise from north) at the centre cell of a 3 x 3 window of a digital elevation model, by Horn's third-order finite differences.

$$\frac{\partial z}{\partial x} = \frac{(z_3 + 2z_6 + z_9) - (z_1 + 2z_4 + z_7)}{8\Delta}, \quad \frac{\partial z}{\partial y} = \frac{(z_1 + 2z_2 + z_3) - (z_7 + 2z_8 + z_9)}{8\Delta}, \quad S = \arctan\|\nabla z|$$

symbol		meaning	SI unit
`window`	in	3 x 3 elevations in metres, row 0 to the north, column 0 to the west	—
`cell_size`	in	cell size (square cells)	m
`slope_deg`	out	slope (degrees)	—
`slope_percent`	out	slope (per cent)	—
`aspect_deg`	out	aspect: downslope direction clockwise from north (none for a flat window)	—

Applies when: square cells; row 0 is the northern row.

```
$ geospatial-analytics solve geo_slope_aspect_horn "window=[[37.2, 42, 47.2], [45.2, 50, 55.2],
[53.2, 58, 63.2]]" "cell_size=10 m"
slope_deg = 43.33171975
slope_percent = 94.33981132
aspect_deg = 327.9946168
  by: Slope and aspect of a 3x3 elevation window (Horn's method)  (Horn (1981): dz/dx = ((c + 2f +
i) - (a + 2d + g))/(8 dx), dz/dy = ((a + 2b + c) - (g + 2h + i))/(8 dy) with y north; slope =
atan(|grad|), aspect = atan2(-dz/dx, -dz/dy))
  cell_size = 10 m   []
  assumes: square cells; row 0 is the northern row
```

What the worked example means: Horn's method estimates the terrain gradient using a three by three window of elevations and a square cell size. The partial derivatives are computed from weighted differences of the nine elevation values divided by eight times the cell size. These gradients determine the steepness as the arctangent of their magnitude and the downslope direction via the two argument arctangent function. For the given inputs, the resulting slope is 43.3317 degrees and the aspect points to 327.995 degrees clockwise from north.

Terrain curvature: total, profile and plan curvature (Zevenbergen-Thorne) — `geo_terrain_curvature`

The total, profile (along the slope) and plan (across the slope) curvature at the centre of a 3 x 3 elevation window from the Zevenbergen-Thorne partial quartic surface.

$$K = -2(D+E), \quad K_{prof} = \frac{-2(DG^2 + EH^2 + FGH)}{G^2 + H^2}, \quad K_{plan} = \frac{2(DH^2 + EG^2 - FGH)}{G^2 + H^2}$$

symbol		meaning	SI unit
`window`	in	3 x 3 elevations in metres, row 0 to the north	—
`cell_size`	in	cell size	m
`curvature`	out	total curvature -2(D + E) (1/m; positive upwardly convex)	—
`profile`	out	profile curvature (1/m)	—
`plan`	out	plan curvature (1/m)	—

Applies when: non-zero gradient at the centre for profile and plan curvature.

```
$ geospatial-analytics solve geo_terrain_curvature "window=[[10, 12, 14], [9, 10, 13], [7, 9, 11]]"
"cell_size=1 m"
curvature = -3
profile = -1.64
plan = 1.36
  by: Terrain curvature: total, profile and plan curvature (Zevenbergen-Thorne)  (Zevenbergen and
Thorne (1987): D, E, F, G, H from the window; profile = -2(DG^2 + EH^2 + FGH)/(G^2 + H^2), plan =
2(DH^2 + EG^2 - FGH)/(G^2 + H^2) (sign convention of Moore et al. 1991 and ArcGIS, without the
factor 100))
  cell_size = 1 m   []
  assumes: non-zero gradient at the centre for profile and plan curvature
```

What the worked example means: This calculation determines the geometric shape of the land surface at the center of the elevation window using the Zevenbergen-Thorne method. The total curvature, reported as -3, indicates whether the point is convex or concave overall. The profile curvature of -1.64 describes the

acceleration or deceleration of flow along the slope direction, while the plan curvature of 1.36 shows how the slope diverges or converges across the contour lines. These values help predict water accumulation and erosion potential based on local surface geometry.

Terrain ruggedness index (Riley), topographic position index and local relief — `geo_terrain_ruggedness`

Riley's terrain ruggedness index (root of the summed squared elevation differences to the 8 neighbours), the topographic position index (centre minus neighbour mean) and the local relief of a 3 x 3 window.

$$TRI = \sqrt{\sum_{i=1}^{8} (z_i - z_0)^2}, \quad TPI = z_0 - \frac{1}{8}\sum_{i=1}^{8} z_i$$

symbol		meaning	SI unit
`window`	in	3 x 3 elevations in metres	—
`TRI`	out	terrain ruggedness index (m)	—
`TPI`	out	topographic position index (m)	—
`relief`	out	maximum minus minimum elevation (m)	—

Applies when: Riley et al. (1999) form of the TRI.

```
$ geospatial-analytics solve geo_terrain_ruggedness "window=[[10, 12, 14], [9, 10, 13], [7, 9,
11]]"
TRI = 6.403124237
TPI = -0.625
relief = 7
  by: Terrain ruggedness index (Riley), topographic position index and local relief  (TRI =
sqrt(sum (z_i - z_0)^2); TPI = z_0 - mean(z_i); relief = max - min)
  assumes: Riley et al. (1999) form of the TRI
```

What the worked example means: This analysis calculates three key metrics for a specific location using a three by three elevation window. The terrain ruggedness index, resulting in 6.40312, measures how uneven the immediate surroundings are compared to the center point. A higher value indicates steeper or more broken terrain. The topographic position index of -0.625 shows whether this spot is relatively high or low; positive values suggest a ridge or peak, while negative values indicate a valley or depression. Finally, the local relief of 7 represents the difference between the highest and lowest elevations within that small neighborhood, giving a sense of the vertical variation present.

Watershed (catchment) of an outlet cell on a small DEM — `geo_watershed_delineation`

The cells that drain to a chosen outlet cell under D8 flow directions: their number, the catchment area and the catchment mask.

$$W(o) = \{c : c \to \cdots \to o\}$$

symbol		meaning	SI unit
`dem`	in	elevations (row 0 to the north), at most 900 cells	—
`cell_size`	in	cell size	m

symbol		meaning	SI unit
`row`	in	outlet row (from 0, north)	—
`col`	in	outlet column (from 0, west)	—
`cells`	out	number of cells in the catchment (outlet included)	—
`area`	out	catchment area	m^2
`mask`	out	1 for cells in the catchment, 0 elsewhere (matrix)	—

Applies when: D8 directions as in geo_d8_flow_direction.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This calculation identifies all land area that drains to the specified outlet cell. Using D8 flow directions, it traces upstream from the point defined by row 3 and column 3. The result includes the total count of contributing cells, which is 16, and the physical surface area of this region, calculated as 1600 m^2. This spatial extent represents the full watershed boundary for runoff accumulation at that specific location.

Urban geography (6)

Huff model: probabilities of shopping at each store from an origin (network distances) — `geo_huff_model`

The Huff probabilities that a consumer at an origin node patronizes each store, from store attractiveness (floor space) and shortest-path distances on a road network.

$$P_{ij} = \frac{S_j^{\alpha} d_{ij}^{-\beta}}{\sum_k S_k^{\alpha} d_{ik}^{-\beta}}$$

symbol		meaning	SI unit
`edges`	in	undirected network as weighted edges between nodes numbered 0 .. n-1	—
`origin`	in	origin node	—
`stores`	in	store nodes	—
`attractiveness`	in	attractiveness of each store (e.g. floor area)	—
`alpha`	in	attractiveness exponent	—
`beta`	in	distance-decay exponent	—
`probabilities`	out	probability of each store	—
`distances`	out	shortest-path distance to each store	—

Applies when: every store reachable and not at the origin.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This analysis determines the likelihood that a consumer at the specified origin will visit each available store. Applying the Huff model weighs each store's attractiveness against its shortest-path distance on the road network. The resulting probabilities indicate where customers are most

likely to shop. For instance, the store with the highest calculated probability shows a value of [0.640296, 0.245874, 0.11383], reflecting its competitive position given its size and accessibility.

Rank-size rule (Zipf) fit of a settlement system — `geo_rank_size_rule`

The Zipf exponent q and fitted size of the largest city from a log-log regression of population on rank (P_r = P_1 r^-q), the R^2 of the fit and the two-city primacy ratio.

$$\ln P_r = \ln P_1 - q \ln r$$

symbol		meaning	SI unit
`populations`	in	populations of the settlements (any order)	—
`q`	out	rank-size exponent (1 is Zipf's law)	—
`P1_fit`	out	fitted population of rank 1	—
`r_squared`	out	R^2 of the log-log fit	—
`primacy`	out	largest / second largest population	—

Applies when: positive populations.

```
$ geospatial-analytics solve geo_rank_size_rule "populations=[8400000.0, 3900000.0, 2700000.0,
2300000.0, 1600000.0, 1500000.0, 1400000.0, 1300000.0]"
q = 0.9031022436
P1_fit = 7735648.989
r_squared = 0.9853069024
primacy = 2.153846154
  by: Rank-size rule (Zipf) fit of a settlement system  (sort descending; least squares of ln P on
ln rank)
  assumes: positive populations
```

What the worked example means: This analysis sorts the settlement populations in descending order and performs a least squares regression of the natural logarithm of population against the natural logarithm of rank. The resulting slope is the negative of the Zipf exponent, which is reported as 0.903102. An exponent near one indicates adherence to the ideal rank-size rule, while values significantly different from one suggest deviations in urban hierarchy. The intercept determines the fitted population for the top-ranked city, shown here as 7.73565e+06. Additionally, the coefficient of determination, 0.985307, measures how well the power law explains the observed data, and the primacy ratio, 2.15385, compares the size of the largest city to the second largest.

Residential segregation (evenness): dissimilarity, Gini and Theil entropy indices — `geo_segregation_evenness`

The evenness dimension of residential segregation of a group across areal units (census tracts): Duncan's index of dissimilarity, the segregation Gini index and Theil's entropy index H.

$$D = \frac{\sum_i t_i |p_i - P|}{2TP(1-P)}, \quad G = \frac{\sum_i \sum_j t_i t_j |p_i - p_j|}{2T^2 P(1-P)}, \quad H = \sum_i \frac{t_i (E - E_i)}{ET}$$

symbol		meaning	SI unit
`x`	in	group count in each areal unit	—
`t`	in	total population of each areal unit	—
`dissimilarity`	out	index of dissimilarity D	—
`gini`	out	Gini segregation index	—
`theil_h`	out	Theil's information theory index H	—

Applies when: 0 <= x <= t in each unit; the group present but not the whole population.

```
$ geospatial-analytics solve geo_segregation_evenness "x=[100, 200, 50, 800]" "t=[1000, 1000, 1000, 1000]"
dissimilarity = 0.6254767353
gini = 0.7170099161
theil_h = 0.3647240802
  by: Residential segregation (evenness): dissimilarity, Gini and Theil entropy indices  (Massey and Denton (1988): D = sum t_i |p_i - P| / (2TP(1-P)); G = sum_ij t_i t_j |p_i - p_j| / (2T^2P(1-P)); H = sum t_i (E - E_i)/(ET))
  assumes: 0 <= x <= t in each unit; the group present but not the whole population
```

In the worked example, x = [100, 200, 50, 800], t = [1000, 1000, 1000, 1000] go through the equation above and give dissimilarity = 0.6254767353, gini = 0.7170099161, theil_h = 0.3647240802. It applies only when 0 <= x <= t in each unit; the group present but not the whole population: outside that it says what is wrong instead of returning a number.

Residential segregation (exposure): isolation, interaction and correlation ratio — `geo_segregation_exposure`

The exposure dimension of residential segregation: the isolation index xPx (probability that a group member meets a member of the same group in their unit), the interaction (exposure) index xPy and the correlation ratio eta^2.

$$ {}_xP^*_x = \sum_i \frac{x_i}{X}\frac{x_i}{t_i}, \quad {}_xP^*_y = \sum_i \frac{x_i}{X}\frac{y_i}{t_i}, \quad \eta^2 = \frac{{}_xP^*_x - P}{1 - P} $$

symbol		meaning	SI unit
`x`	in	group count in each areal unit	—
`t`	in	total population of each areal unit	—
`isolation`	out	isolation index xPx	—
`exposure`	out	interaction index xPy	—
`correlation_ratio`	out	correlation ratio (xPx - P)/(1 - P)	—

Applies when: 0 <= x <= t in each unit; the group present but not the whole population.

```
$ geospatial-analytics solve geo_segregation_exposure "x=[100, 200, 50, 800]" "t=[1000, 1000, 1000, 1000]"
isolation = 0.602173913
exposure = 0.397826087
correlation_ratio = 0.4416475973
  by: Residential segregation (exposure): isolation, interaction and correlation ratio  (xPx = sum (x_i/X)(x_i/t_i); xPy = sum (x_i/X)(y_i/t_i) with y = t - x; eta^2 = (xPx - P)/(1 - P))
  assumes: 0 <= x <= t in each unit; the group present but not the whole population
```

What the worked example means: This calculation determines residential segregation using exposure metrics. It computes the isolation index, which represents the probability that a member of the group encounters another member of the same group within their local area. The result is 0.602174. Simultaneously, it calculates the interaction or exposure index, showing the likelihood of encountering members of the other group, resulting in 0.397826. Finally, the correlation ratio is derived to measure the variance explained by grouping, yielding 0.441648. These values help interpret whether groups share neighborhoods or live apart.

Spatial interaction (gravity) model fitted by Poisson regression — `geo_gravity_model_poisson_fit`

The constant k, the origin and destination mass exponents and the distance-decay exponent of the unconstrained gravity model T = k O^alpha D^beta d^-gamma, fitted to observed flows by Poisson maximum likelihood.

$$T_{ij} \sim \mathrm{Poisson}(\mu_{ij}), \quad \ln \mu_{ij} = \ln k + \alpha \ln O_i + \beta \ln D_j - \gamma \ln d_{ij}$$

symbol		meaning	SI unit
`T`	in	observed flows between the pairs	—
`O`	in	origin masses (e.g. populations)	—
`D`	in	destination masses	—
`d`	in	distances between the pairs	—
`k`	out	constant	—
`alpha`	out	origin mass exponent	—
`beta`	out	destination mass exponent	—
`gamma`	out	distance-decay exponent	—

Applies when: non-negative flows, positive masses and distances that vary.

Not in `geospatial-analytics` 1.1.0 yet: use the Python package or the model's page in the browser.

What the worked example means: This analysis estimates the parameters of a spatial interaction model by treating observed flows as count data. The resulting constant 0.000370957 scales the overall magnitude of interactions, while the exponents 0.802892 and 0.33475 quantify how sensitive flows are to changes in origin and destination masses respectively. The parameter 0.509079 describes the rate at which interaction volume diminishes with increasing distance between locations.

Urban population density gradient (Clark's negative exponential model) — `geo_density_gradient`

The central density D0, density gradient b and half-distance of Clark's model D(r) = D0 exp(-b r) fitted by log-linear least squares to densities at distances from the centre, with the R^2.

$$D(r) = D_0 e^{-br}, \quad \ln D = \ln D_0 - br$$

symbol		meaning	SI unit
`r`	in	distances from the centre (any unit)	—
`D`	in	population densities at those distances	—
`D0`	out	fitted central density	—
`b`	out	density gradient (per distance unit)	—
`r_squared`	out	R^2 of the log-linear fit	—
`half_distance`	out	distance over which density halves (ln 2 / b), when b > 0	—

Applies when: positive densities; distances not all equal.

```
$ geospatial-analytics solve geo_density_gradient "r=[1, 2, 4, 6, 8, 10]" "D=[9000, 7600, 5300,
3900, 2800, 2000]"
D0 = 10546.36506
b = 0.1663827336
r_squared = 0.9996240123
half_distance = 4.165980239
  by: Urban population density gradient (Clark's negative exponential model)  (least squares of ln
D on r)
  assumes: positive densities; distances not all equal
```

What the worked example means: This analysis fits Clark's negative exponential model to urban population densities at varying distances from the city center. By applying least squares regression to the natural logarithm of density against distance, it estimates the central density as 10546.4 and the rate of decline as 0.166383. The half-distance, where density drops by fifty percent, is calculated as 4.16598. The goodness of fit is indicated by an R-squared value of 0.999624, showing how well the single-center assumption explains the observed data.

Questions and reports: support@ai999x.com. AI999X — The Future Is Self-Improving.